



國立臺灣科技大學
電 機 工 程 系

碩士學位論文

學號: M11007445

權重曉知與降低位元精確度
之低成本 AI 加速器架構設計

**Weight-Aware and Reduced-Precision
Architecture Designs for Low-Cost AI
Accelerators**

研究生: 黃煜翔

指導教授: 呂學坤 博士

中華民國 113 年 月 日

致謝

在研究所經過了兩年的努力，終於完成了碩士論文，自從成為專題生後，就開始進入實驗室學習，到現在已經學到了許多寶貴的知識與經驗，非常謝謝呂學坤教授的指導與照顧，不僅提供經費讓我參加半導體研究中心的訓練課程和超大型積體電路設計的研討會，同時還提供實驗室的各種資源，不論是數位電路的合成、繞線和佈局軟體等等，對我來說都是不可或缺的實作經驗。在開會時，教授每一次都會在論文上給予一些方向與啟發，讓我們從中去思考任何的創意與可能性，同時教導我們做簡報的技巧與品質，以及如何將報告的核心重點說給觀眾聽，在研究所的這段期間，讓我獲得了很大的成長與進步。

謝謝實驗室的學長姊們，耀安、承佑、幫心和若綺，在我剛進入實驗室時，教導我軟體工具的使用，當我在專業領域有疑惑時，也願意替我解答，讓我獲益良多。在此謝謝實驗室的夥伴們，允中、義正和百濤，因為有你們陪在我身邊，我才能度過重重難關，當研究過程有問題時，都能夠互相討論並適時的給予意見與幫助，也謝謝學弟妹們，俊凱、珮蓉和炫吟，有你們的加入與關心，我才能夠獲得信心與勇氣，去完成每一項作業與研究。

最後感謝我的家人們，時常關心我的課業情況，在我面對壓力的時候，給予我鼓勵與支持，讓我有繼續堅持下去的動力，也謝謝這一路上與我分享經驗的好朋友們。

摘要

由於深度神經網路 (Deep Neural Network, DNN) 快速的發展，至今已被廣泛應用於各種任務，例如語音辨識、圖像辨識和自動駕駛等等，模型的訓練與推論 (Inference) 需要大量的運算，這就促使了對於專用硬體加速器的需求，以滿足深度學習應用的高效能運算，谷歌 (Google) 科技公司為了提高硬體的計算效率而設計的張量處理器 (Tensor Processing Unit, TPU) 就是一種硬體加速器 [1]，給神經網路提供了良好的優化。

本篇論文提出的降低位元精確度之張量處理器 (Reduced Precision TPU, RPTPU)，主要是透過權重值 (Weight) 的特性來針對脈動陣列 (Systolic Array) 內部的處理單元 (Processing Element, PE) 進行面積成本的優化，本篇提出的方法可以將權重值的精確度 (Precision) 縮減，同時不使深度神經網路模型的準確度 (Accuracy) 大幅降低，藉由縮減輸入資料的精確度，將能使用更小的乘法器 (Multiplier) 進行乘法運算，以降低張量處理器的硬體成本。

本篇論文提出了第一型態的降低位元精確度之張量處理器 (Type-1 RPTPU) 與第二型態的降低位元精確度之張量處理器 (Type-2 RPTPU) 的架構，硬體成本與傳統的張量處理器相比分別下降了 34 % 與 43 %，而更小的乘法器可以加快硬體的計算速度，其操作頻率的效能 (Performance) 也提升了 1.15 倍至 1.19 倍，且功率消耗也能降低 18 % 至 27 %。本篇論文與其他相關論文 [2, 3] 相比之下，其操作頻率也能提升將近 1.15 倍以上的效能，且降低 10 % 至 34 % 的功率消耗，在 MLP 模型的準確度相比之下，本篇論文在相差不遠的硬體成本中準確度也高了 3 % 以上。

Abstract

Due to the rapid development of Deep Neural Networks (DNN), they have been widely applied in various tasks such as speech recognition, image recognition, and autonomous driving, etc. The training and inference of DNN models require extensive computations, leading to a demand for specialized hardware accelerators to meet the high computation needs for deep learning applications. Tensor Processing Unit (TPU) is a dedicated hardware accelerator designed by Google to improve hardware computation efficiency [1], providing optimizations for neural networks.

This thesis presents the Reduced Precision Tensor Processing Unit (RPTPU), which aims to optimize the area cost of processing elements (PEs) within the systolic array-based TPU by leveraging the characteristics of weight values. The proposed techniques reduce the precision of weight values without significant accuracy drops. By reducing the precision of the input data, smaller multipliers can be used for multiplication operations, thereby lowering the hardware cost of the tensor processing unit. Two types of Reduced Precision Tensor Processing Units, namely Type-1 RPTPU and Type-2 RPTPU, are proposed in this thesis. The hardware costs are reduced by 34% and 43%, respectively as compared to the traditional tensor processing unit. By using smaller multipliers can also accelerate the hardware computation speed from 1.15 to 1.19 times. Additionally, power consumption can be reduced from 18% to 27%. As compared to other relevant papers [2] and [3], this paper achieves an improvement in performance by almost 1.15 times and a reduction in power consumption by 10% and 34%, respectively. In terms of accuracy for MLP models, this thesis can increase 3% inference accuracy with similar hardware costs.

目錄

致謝	i
摘要	ii
Abstract	iii
目錄	iv
圖目錄	vi
表目錄	x
第一章 簡介	1
1.1 背景與動機	1
1.2 組織架構	5
第二章 深度神經網路之基本原理	6
2.1 神經元與深度神經網路原理與架構	6
2.2 全連結神經網路	10
2.3 卷積神經網路	11
第三章 張量處理器 (Tensor Processing Unit, TPU)	17
3.1 張量處理器之原理	17
3.2 張量處理器之架構	18
3.3 張量處理器之操作流程	20
3.3.1 權重固定之資料流	21
3.3.2 輸出固定之資料流	25
3.4 降低成本之張量處理器之相關研究	28

3.4.1 動態範圍之無偏乘法器 [2].....	28
3.4.2 近似計算之張量處理器 [3].....	30
第四章 降低位元精確度之張量處理器 (Reduced Precision TPU, RPTPU).....	34
4.1 權重值分佈之觀察.....	34
4.2 權重值預處理之方式	35
4.3 降低位元精確度之張量處理器原理	37
4.4 降低位元精確度之張量處理器架構	38
4.4.1 第一型態的降低位元精確度之張量處理器架構 (Type-1 RPTPU).....	39
4.4.2 第二型態的降低位元精確度之張量處理器架構 (Type-2 RPTPU).....	42
第五章 實驗結果	46
5.1 深度學習模型設定.....	46
5.2 準確度分析	47
5.3 硬體成本分析	50
5.4 張量處理器之數位電路模擬	51
5.5 超大型積體電路實現	55
第六章 結論與未來展望.....	57
6.1 結論.....	57
6.2 未來展望	57
參考文獻.....	58

圖目錄

圖 1.1 神經網路示意圖.....	1
圖 1.2 重要的連續位元之示意圖 [12].....	4
圖 2.1 單一個神經元之示意圖.....	6
圖 2.2 Sigmoid 函數.....	7
圖 2.3 雙曲正切函數.....	8
圖 2.4 線性整流函數.....	9
圖 2.5 深度神經網路示意圖.....	10
圖 2.6 全連結神經網路示意圖.....	10
圖 2.7 卷積神經網路示意圖.....	11
圖 2.8 卷積運算範例一.....	12
圖 2.9 卷積運算範例二.....	12
圖 2.10 卷積運算範例三.....	13
圖 2.11 卷積運算範例四.....	13
圖 2.12 圖形縮放示意圖.....	14
圖 2.13 池化層運算範例一.....	14
圖 2.14 池化層運算範例二.....	15
圖 2.15 池化層運算範例三.....	15
圖 2.16 池化層運算範例四.....	16

圖 3.1 張量處理器之運算原理示意圖.....	17
圖 3.2 單一個處理單元之示意圖.....	18
圖 3.3 矩陣乘法器之架構示意圖.....	19
圖 3.4 張量處理器之架構示意圖 [1].....	20
圖 3.5 二維輸入矩陣轉換成一維輸入陣列之示意圖.....	21
圖 3.6 脈動陣列之卷積運算範例一.....	22
圖 3.7 脈動陣列之卷積運算範例二.....	22
圖 3.8 脈動陣列之卷積運算範例三.....	23
圖 3.9 脈動陣列之卷積運算範例四.....	24
圖 3.10 脈動陣列之卷積運算範例五.....	24
圖 3.11 矩陣運算之示意圖.....	25
圖 3.12 脈動陣列之矩陣運算範例一.....	25
圖 3.13 脈動陣列之矩陣運算範例二.....	26
圖 3.14 脈動陣列之矩陣運算範例三.....	27
圖 3.15 脈動陣列之矩陣運算範例四.....	27
圖 3.16 縮短資料字長之示意圖 [2].....	28
圖 3.17 無偏乘法器之乘法計算示意圖 [2].....	29
圖 3.18 無偏乘法器之硬體架構 [2].....	30
圖 3.19 預近似單元之硬體架構 [3].....	31

圖 3.20 近似處理單元之硬體架構 [3].....	32
圖 3.21 近似張量處理器之硬體架構 [3].....	33
圖 4.1 權重之 MSR 分佈圖	34
圖 4.2 權重縮減之示意圖.....	36
圖 4.3 權重預處理單元之架構.....	37
圖 4.4 權重縮減後之運算示意圖.....	38
圖 4.5 第一型態的降低位元精確度之處理單元之示意圖.....	40
圖 4.6 第一型態的降低位元精確度之張量處理器之示意圖.....	41
圖 4.7 精確度 16 位元的權重中最低的 9 位元資料累積分佈.....	42
圖 4.8 第二型態的降低位元精確度之處理單元之示意圖.....	43
圖 4.9 第二型態的降低位元精確度之張量處理器之示意圖.....	44
圖 4.10 降低位元精確度之張量處理器架構.....	45
圖 5.1 第一型態的降低位元精確度之張量處理器準確度分析.....	48
圖 5.2 第二型態的降低位元精確度之張量處理器準確度分析.....	49
圖 5.3 第一型態的降低位元精確度之處理單元之電路模擬.....	52
圖 5.4 第一型態的降低位元精確度之張量處理器之電路模擬範例一.....	52
圖 5.5 第一型態的降低位元精確度之張量處理器之電路模擬範例二.....	53
圖 5.6 第二型態的降低位元精確度之處理單元之電路模擬.....	53
圖 5.7 第二型態的降低位元精確度之張量處理器之電路模擬範例一.....	54

圖 5.8 第二型態的降低位元精確度之張量處理器之電路模擬範例二.....	54
圖 5.9 第二型態的降低位元精確度之張量處理器之佈局圖.....	56

表目錄

表 4.1 權重之 MSR 分佈表	35
表 5.1 神經網路模型之參數.....	47
表 5.2 第一型態的降低位元精確度之張量處理器之準確度參數.....	48
表 5.3 第二型態的降低位元精確度之張量處理器之準確度參數.....	49
表 5.4 與其他相關論文之準確度比較.....	50
表 5.5 與其他相關論文之綜合結果比較.....	51
表 5.6 張量處理器的超大型積體電路實現結果.....	55

第一章 簡介

1.1 背景與動機

隨著時代不斷的發展和演進，人們對於人工智慧 (Artificial Intelligence, AI) 與機器學習演算法的研究越來越受關注，隨著計算機硬體性能的提升，人們得以處理更複雜的數據和算法，這促使了對於模擬人類思維的構想，隨著數據量的急劇增加，人們開始思考如何有效地處理、分析和利用這些大量的數據，許多科技公司為了解決這些複雜的問題，致力於研究人工智慧，雖然先前的人工智慧與機器學習的技術不夠成熟，導致關注度有些微的衰退，但是隨著深度學習技術的不斷發展與進步，越來越多的應用場景需要處理龐大的數據集和複雜的模型結構。處理龐大的神經網絡 (Neural Networks, NNs) 模型所需的計算資源大幅增加，而其結構如圖 1.1 所示，輸出層 (Output Layer) 的每個神經元 (Neuron) 都將透過其對應連線的輸入層 (Input Layer) 與中間的權重值進行乘積計算，以得到輸出結果。

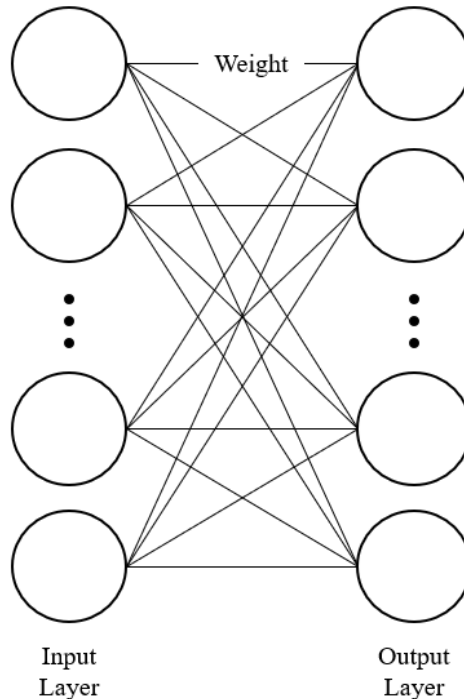


圖 1.1 神經網路示意圖

通用的中央處理器 (Central Processing Unit, CPU) 和專門用來處理圖形或影像的圖形處理器 (Graphics Processing Unit, GPU) 雖然能夠執行這些神經網路的任務，但是對於深度學習來說，仍然存在一些計算效率上的瓶頸。而深度神經網路是一種基於神經網路架構的機器學習模型，具有多層次的神經網路，被廣泛應用於各種任務，包括圖像辨識與自然語言處理 (Natural Language Processing, NLP)，模型的訓練與推論需要大量的運算，這對於傳統的中央處理器來說有資料儲存、資料搬遷和處理速度的問題，因此傳統的通用計算硬體在處理這種規模的任務時，面臨著性能瓶頸和能源效率的問題 [4]，這就促使了對於專用硬體加速器的需求增加，以滿足深度學習應用的高效能運算。

許多公司近年來致力於研發各種硬體加速器，特別是專門用於提高人工智慧的性能與效率的硬體，谷歌科技公司為了提高硬體的計算效率而設計的張量處理器就是一種人工智慧加速器，給神經網路提供了良好的優化，深度神經網路中的主要運算為矩陣乘法與累加運算，谷歌科技公司的張量處理器由 256×256 的矩陣乘法器 (Matrix Multiplication Unit, MMU) 所組成 [1]，矩陣乘法器裡面是透過眾多的處理單元來進行乘積與累加運算，並且透過記憶體與處理單元之間的交錯 (Interleaving) 來運行，處理器的基本架構為串流 (Streaming) 架構與單核 (Single Core) 架構，串流架構包含大量的處理核心且每個核心都能夠同時處理不同的數據，同時強調大規模平行 (Parallel) 處理，這對於圖形處理的任務非常重要，因為許多圖形和影像操作可以平行進行；而單核架構只包含單個處理核心，每個處理核心都能夠獨立執行指令，但在同一時刻只能處理一個任務，通常應用於較簡單的任務或計算要求不高的場景，例如嵌入式系統與低功耗設備。而我們的脈動陣列就是其中一種串流架構，主要以處理單元組成 [5]，處理單元內部主要由組合邏輯來執行乘法與累加的運算，處理單元會將記憶體傳送進來的激活值 (Activation) 與權重值相乘並與部分和 (Partial Sum) 相加，每個處理單元都有一個暫存器來儲存部分和，直到所有部分和累加完成後即為輸出的結果。一個大型

的加速器通常會有將近數萬個處理單元進行連接，而加速器普遍都會在伺服器端進行神經網路運算，由於大多數的資料都是由客戶端所產生，所以邊緣化人工智慧加速器的需求也逐漸增加，邊緣運算提供一個低成本且只需要較低的資料傳輸速度即可達到較快的反應時間 [6, 7]，且深度神經網路已經趨近成熟，許多知名科技公司開始發展將神經網路應用於邊緣運算裝置上，像是 NVIDIA [8]、Microsoft [9] 和 Tesla [10]，都已經有特定的加速器運用在邊緣運算的裝置上，且這些加速器的面積與功耗都很低。

在深度學習的模型中，對於輕微的輸入變化具有一定的容忍度，即使輸出結果有些差異也不會對整體的任務產生影響，例如圖像處理、文字處理和語音辨識等，系統需要在有限時間內快速響應，在這種情況下，依據使用者可以容忍的誤差範圍內，可以犧牲一些計算的精確度來換取更快的推論速度，因此人工智慧加速器通常不需要非常精確的計算，因為他們主要用於加速特定類型的計算任務，而不是一般的數值計算。在深度學習中，常用的計算包括矩陣乘法與卷積 (Convolution) 運算，這些計算涉及大量相同類型的運算，近似的結果對於一些應用來說也是可以接受的，近似運算是將乘法器或是加法器 (Adder) 中輸入資料的精確度縮減，不僅可以避免輸出結果的誤差太大，同時又能節省面積成本與能量消耗 [11]。

本篇論文主要是透過權重的特性來針對脈動陣列內部的處理單元進行面積成本的優化，由於深度學習模型中訓練好的權重值普遍接近零，當這些權重值轉換成二進制的定點數 (Fixed-Point) 格式後，從高位元往低位元會產生連續性的 0 或連續性的 1，我們將其稱作重要的連續位元 (Most-Significant Run, MSR) [12]，如圖 1.2 所示。

0.004096	-0.420437	0.153534	-0.197340	00000001	11001010	00010100	11100111
-0.114851	0.036513	-0.028751	0.189524	11110001	00000101	11111100	00011000
0.174329	-0.028819	0.143463	0.159553	00010110	11111100	00010010	00010100
-0.243178	0.036531	-0.082088	0.139659	11100001	00000101	11110101	00010010
0.025325	-0.313233	0.173153	-0.173304	00000011	11010111	00010110	11101010
0.010150	-0.004496	-0.362920	-0.080113	00000001	11111111	11111011	11110110
-0.280939	0.172913	-0.134351	-0.217531	11011100	00010110	11101111	11100100
-0.106031	-0.042388	-0.109121	0.117359	11110010	11111011	11110010	00001111
Original Data				Fixed Point			

圖 1.2 重要的連續位元之示意圖 [12]

假如一個權重值從高位元算起有 4 個重要的連續位元，也就是出現了 4 個連續的位元 0 或位元 1，那我們將可以用 1 個符號位元 (Sign Bit) 來取代這 4 個連續位元，並且不會造成準確度的降低。既然權重值可以藉由重要的連續位元的特性來讓字長 (Word Length) 縮短，那麼在張量處理器中的脈動陣列裡，每個處理單元內部將可以使用更小的乘法器來計算結果，使整體的面積與能量消耗減少，更短的輸入字長還可以使乘法器更快速的計算輸出結果，提升整體的效能。

最後本篇論文以 MLP [13]、LeNet [14]、ResNet [15]、AlexNet [16] 深度學習模型來進行實驗並分析結果。實驗結果顯示本篇論文的降低位元精確度之張量處理器，由於大部分的權重值可以在不造成準確度下降的情況下縮減字長，因此在 MLP 模型下的準確度跟傳統的張量處理器相比只下降了 2 % 以內，縮減字長也使得張量處理器中的處理單元內部可以使用更小的乘法器，因此第一型態的降低位元精確度之張量處理器與第二型態的降低位元精確度之張量處理器的硬體成本跟傳統的張量處理器相比分別下降了 34 % 與 43 %，而更小的乘法器可以加快硬體的計算速度，其操作頻率的效能也提升了 1.15 倍至 1.19 倍，且功率消耗也能降低 18 % 至 27 %。本篇論文與其他相關論文 [2, 3] 相比之下，其操作頻率也能提升將近 1.15 倍以上的效能，且降低 10 % 至 34 % 的功率消耗，在 MLP 模型的準確度相比之下，本篇論文在相差不遠的硬體成本中準確度也高了 3 % 以上。

1.2 組織架構

本篇論文章節內容如下，第二章將介紹深度神經網路之基本原理，包括深度神經網路的架構以及全連結神經網路與卷積神經網路。第三章將介紹張量處理器之基本原理，並說明張量處理器如何達到高效能以加速深度學習的計算，同時會介紹張量處理器的硬體架構。接下來第四章將介紹第一型態與第二型態的降低位元精確度之張量處理器的原理，以張量處理器為基礎進行成本的改良，達到降低成本的效果，同時不會讓準確度嚴重下降，並會介紹兩種型態的降低位元精確度之張量處理器的硬體架構。第五章為實驗結果，介紹模擬的模型種類與環境架構的設置，並分析準確度與硬體成本。第六章為結論以及未來的展望。

第二章 深度神經網路之基本原理

2.1 神經元與深度神經網路原理與架構

深度神經網路是一種基於人工神經元結構的機器學習模型，廣泛應用於圖像處理、語音辨識、自然語言處理等領域。其獨特之處在於它能夠通過多層次的神經元網絡來學習複雜的特徵和模式。深度神經網路的基本架構是神經元，也被稱為節點。每個神經元接收多個輸入，進行線性加權，然後通過激活函數 (Activation Function) 進行非線性轉換後產生輸出。深度神經網路包含多個層次，每一層由多個神經元組成，並且與前一層和後一層的神經元相互連接。通過這種結構，深度神經網路能夠逐層提取特徵，並進一步統整訊息。

在類神經網路中，通常會有數個階層，每個階層中會有數十個到數百個神經元，圖 2.1 為一個神經元的示意圖，一個神經元有 n 個輸入，每個輸入分別乘上對應的權重值後再和偏差值 (Bias) 加總起來，加總後的結果會進行激活函數的轉換，當成神經元的輸出。每個神經元會跟下一層的神經元有特殊的連接關係，使上一層神經元的輸出值經過權重計算後再傳遞給下一層的神經元 [17]。

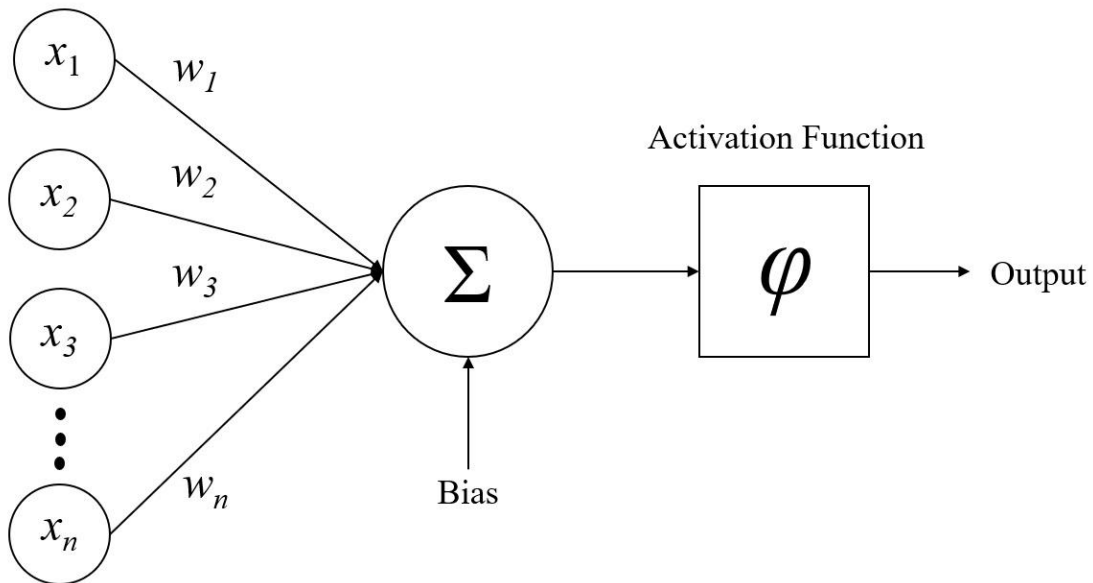
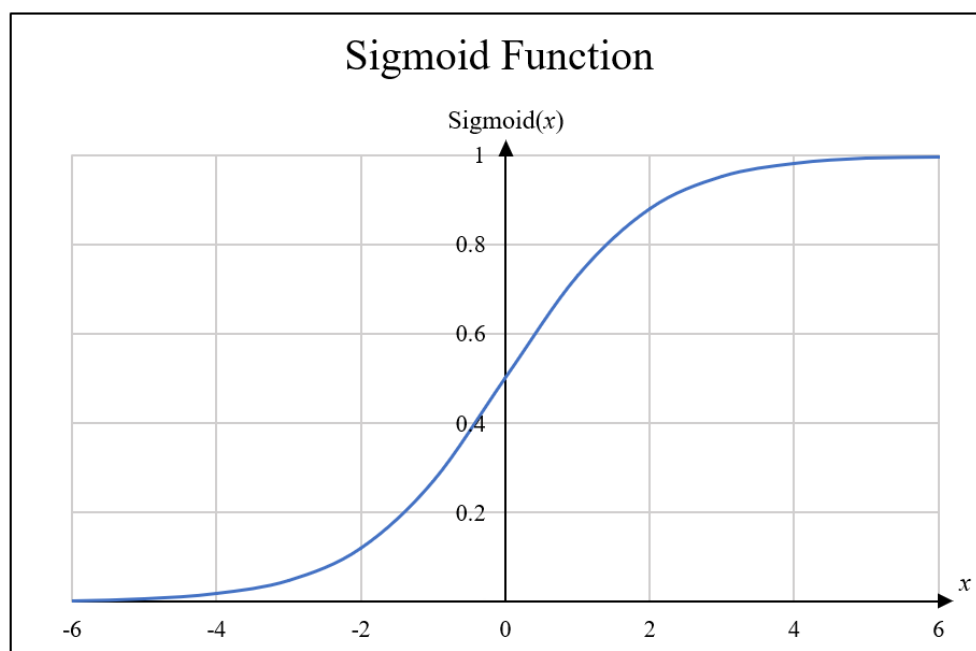


圖 2.1 單一個神經元之示意圖

激活函數在神經網路中扮演著至關重要的角色，激活函數引入了非線性的轉換，使得神經網路能夠學習更加複雜的映射關係。不同的激活函數具有不同的特性，可以適應不同的任務和數據分佈。選擇合適的激活函數可以使模型更好的整合數據與提高性能。要是不使用激活函數，每一層的操作將被視為一個線性轉換，這會使多層的神經網路等效於一層的神經網路，這將會造成類神經網路訓練變得毫無意義。

以下會介紹三種常見的激活函數，圖 2.2 為 S 型 (Sigmoid) 函數的示意圖，面對各種輸入值，S 型函數的輸出只會介於 0 到 1 之間，輸出範圍有限且優化穩定，是一個平滑連續的函數，公式相對簡單，易於理解和實現，這使得它在早期的神經網路研究和應用中獲得了廣泛的應用 [18]。

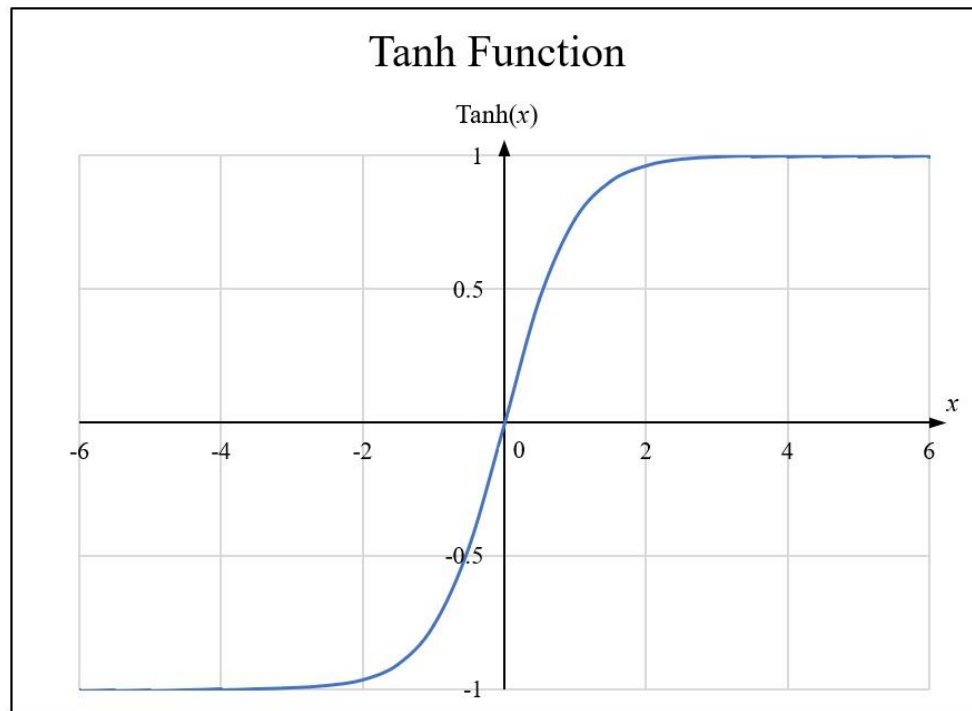


$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

圖 2.2 Sigmoid 函數

另一種常見的函數為雙曲正切 (tanh) 函數 [19]，與 S 型函數類似，雙曲正切函數將輸入映射到一個有限的範圍內，但其範圍介於 -1 到 1 之間，這使得

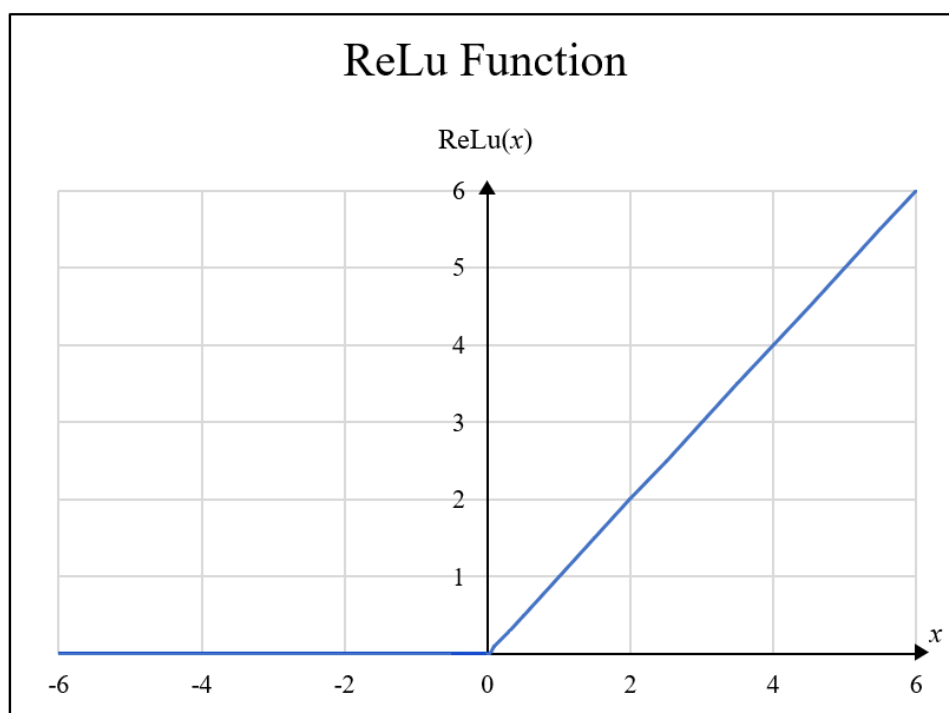
它可以產生負數的輸出，提供更大的表達能力。雙曲正切函數是一個以原點為中心的函數，這意味著它的輸出在平均值附近波動，這對於訓練深度神經網路時有助於更快地收斂，如圖 2.3 所示。



$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

圖 2.3 雙曲正切函數

第三種常見的激活函數為線性整流 (Rectified Linear Unit, ReLU) 函數 [20]，如圖 2.4 所示，線性整流函數在輸入大於 0 的情況下，輸出等於輸入；輸入小於 0 或等於 0 時，輸出等於 0，這種簡單的運算方式使得線性整流的計算效率變得非常高，特別適合用於大型神經網路的訓練。線性整流函數解決了梯度過大的問題，具有良好的計算速度與收斂速度等特性，是近年來最頻繁被使用的激活函數。



$$ReLU(x) = \text{Max}(0, x)$$

圖 2.4 線性整流函數

深度神經網路的架構是一種由多層神經元組成的神經網路模型。它通常由輸入層、多個隱藏層 (Hidden Layer) 和一個輸出層組成，每一層都包含多個神經元。如圖 2.5 所示，最左邊為輸入層，中間為多個隱藏層，右邊為輸出層，每個層之間都以多個權重作為連線，當資料輸入進來後，將與權重做相乘及累加的運算，進行線性組合和非線性轉換，然後將結果傳遞到下一層，經過多層運算後，最後輸出層會進行特定的函數轉換，將輸出的判斷結果產生出來，假如結果辨識正確，即為推論成功；假如結果辨識錯誤，則為推論失敗，而此神經網路的成功辨識機率，我們稱之為準確度，可以用來評估此深度神經網路訓練結果的好壞。

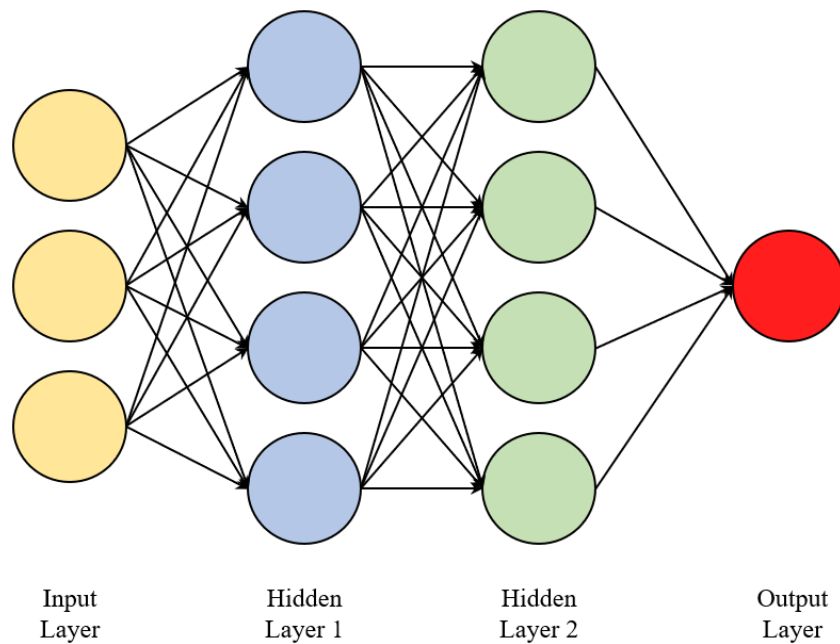


圖 2.5 深度神經網路示意圖

2.2 全連結神經網路

全連結神經網路（Fully Connected Neural Network）是一種基本的神經網路結構 [21]，輸入層有 n 個神經元，輸出層有 m 個神經元，每個神經元都與前一層的所有神經元相連接，也就是每個輸入特徵與每個神經元之間都有一個權重連接，如圖 2.6 所示。

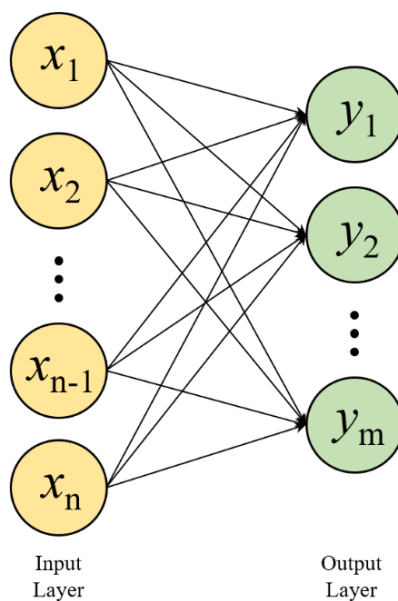


圖 2.6 全連結神經網路示意圖

2.3 卷積神經網路

卷積神經網路 (Convolutional Neural Network, CNN) 是一種特殊的神經網路架構 [22, 23]，由一個或多個卷積層 (Convolutional Layer) 組合而成，主要用於提取圖像中的特徵。如圖 2.7 所示，它通過名為卷積核 (Kernel) 的過濾器在輸入的圖像 (Input Map) 上滑動，將圖像的局部區域與卷積核進行資料相乘與相加，將特徵萃取出來後產生特徵圖 (Feature Map)。輸入的圖像資料經過卷積層後會經過池化層 (Pooling Layer)，池化層用於減少特徵圖的尺寸，同時保留重要的特徵。常用的池化操作包括最大池化 (Max Pooling) 和平均池化 (Average Pooling)，藉由保留每個區域中的最大值或平均值來減少特徵圖的大小，進而減少後續的計算成本。

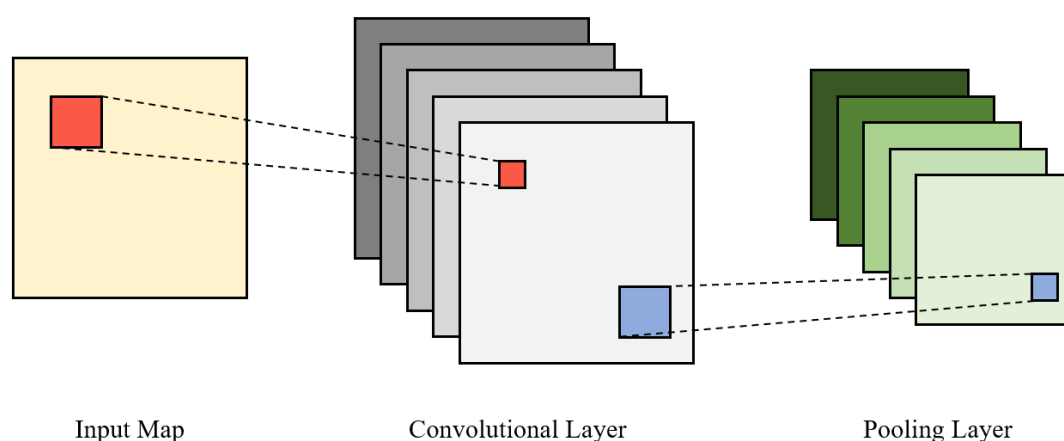


圖 2.7 卷積神經網路示意圖

在卷積神經網路的計算過程中，由於輸入為圖像的資料，因此會有大量的計算資料需要處理，比全連結神經網路花費更多時間，計算的方式與過程如圖 2.8 所示，當我們輸入一張特徵圖，此圖像會經過卷積核做運算，將卷積核對應到的圖像區域進行乘積並相加的動作，計算出的結果即為輸出特徵圖的第一個值，計算結果為 2。

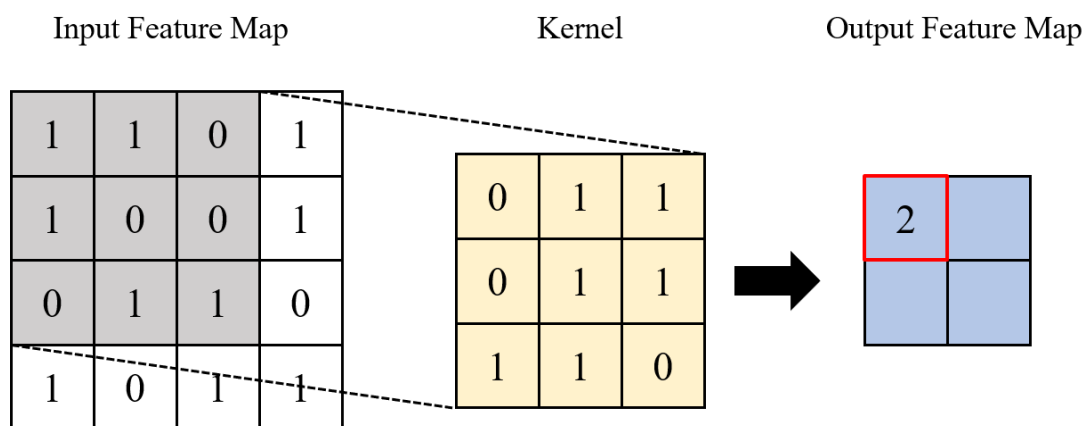


圖 2.8 卷積運算範例一

運算完成後，卷積核會向右移動一個步長 (Stride) 的距離，此範例步長為 1，如圖 2.9 所示，移動一個步長後再將卷積核對應到的新區塊進行乘積並相加，得到輸出結果為 4。

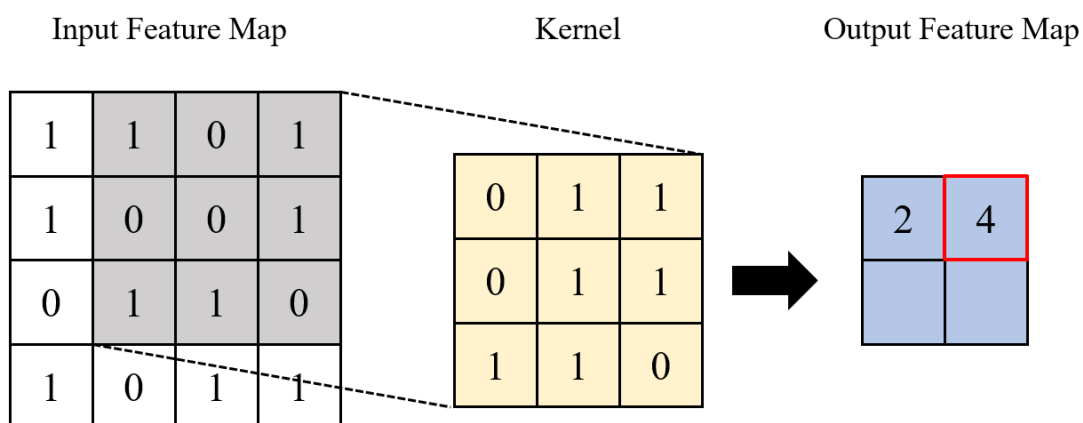


圖 2.9 卷積運算範例二

當卷積核無法再繼續往右移動時，它會回到最左邊並向下移動一個步長的距離，並繼續執行運算，如圖 2.10 所示。

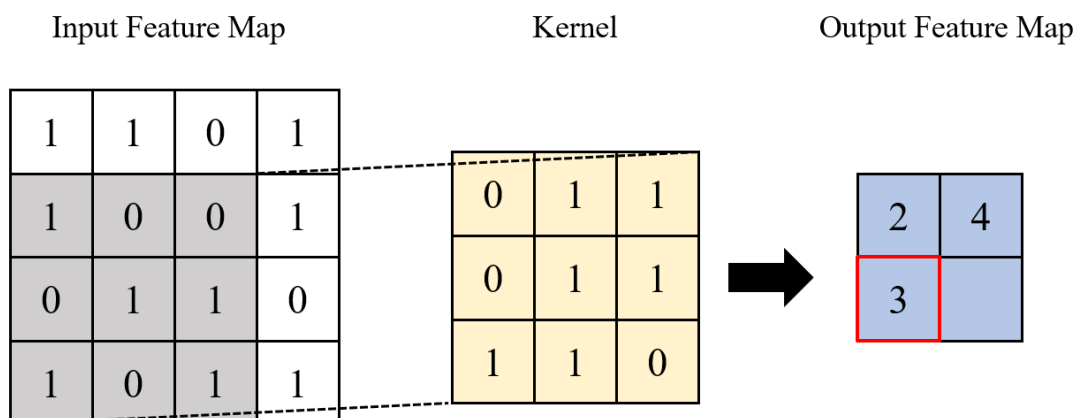


圖 2.10 卷積運算範例三

依此類推，當卷積核已經移動到最右邊，而且也無法繼續往下移動時，即完成所有的計算結果，如圖 2.11 所示，若是有多個卷積核，將會繼續下一個卷積核的運算，並重複上述之運算過程，等到所有的卷積核運算皆完成之後，會將輸出特徵圖傳遞給下一層，變成下一層的輸入資料繼續做運算。

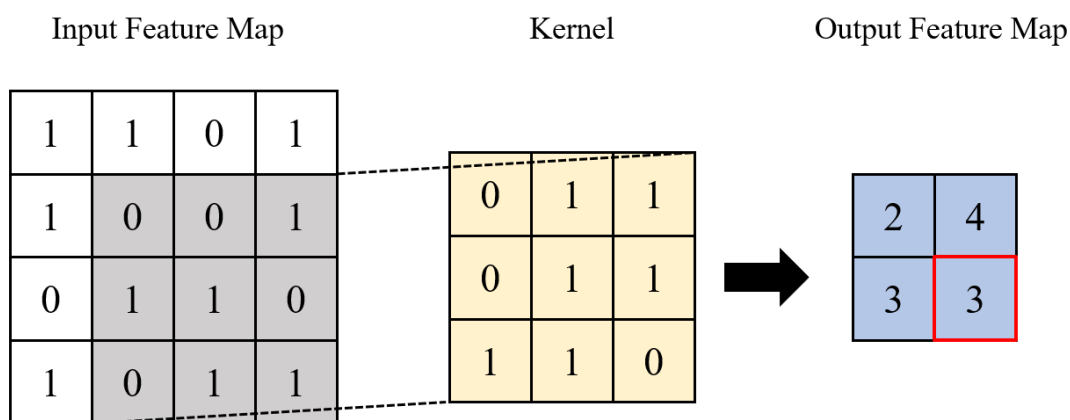


圖 2.11 卷積運算範例四

卷積層的運算完成後，會經過池化層做運算 [24, 25]，池化層的主要概念是當我們在做圖片的特徵萃取的過程中，圖形的縮放並不會嚴重影響到我們圖像辨識的結果，如圖 2.12 所示，經由這樣的縮放我們也可以再一次的減少神經網路的參數。池化層也有一個池化核，也是在輸入圖像上進行滑動並運算，但和卷積核不同的地方是池化核的步長移動的區域通常不會互相覆蓋，否則

會造成最大池化或平均池化後，圖形的縮放效果不佳。

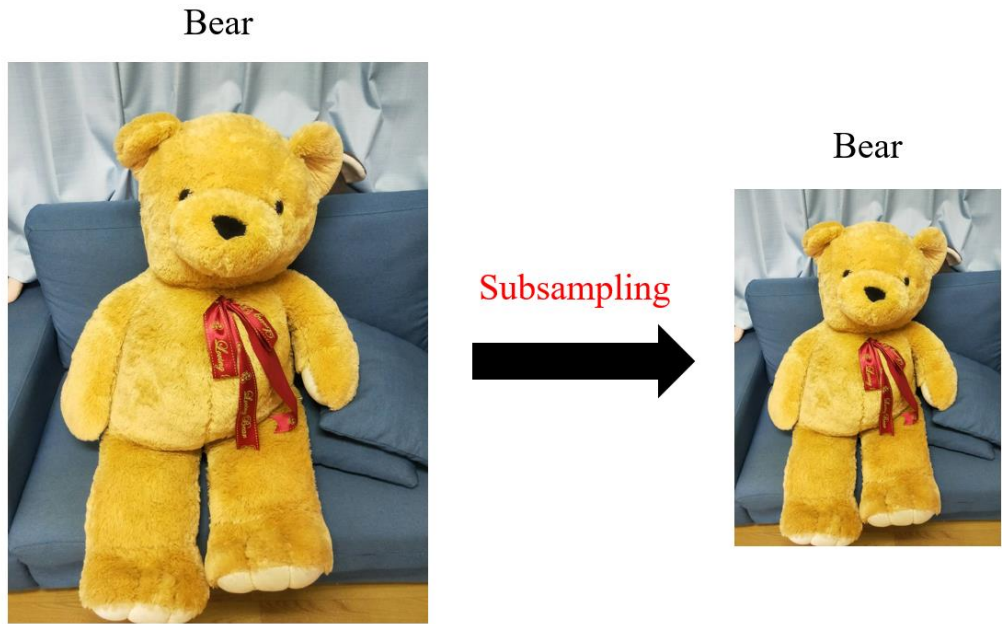


圖 2.12 圖形縮放示意圖

許多研究都發現，最大池化的效果會比平均池化來的好，因此現在大多使用最大池化來做運算，如圖 2.13 所示，此範例為最大池化的過程，當特徵圖輸入進來時，池化核會對應到最左上角的圖像區域，並將圖像裡的值互相比較，將最大的值取出來，得到輸出結果為 7。

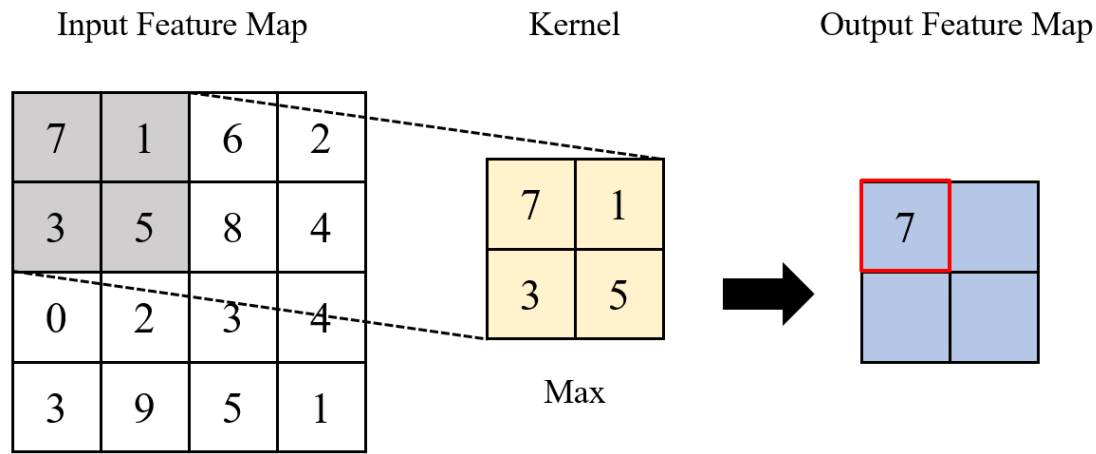


圖 2.13 池化層運算範例一

運算完成後，池化核將會向右移動一個步長，由於滑動後的新區域並不會覆蓋到先前的池化核所對應的區域，因此範例步長為 2，並繼續進行最大池化，得到輸出結果為 8，如圖 2.14 所示。

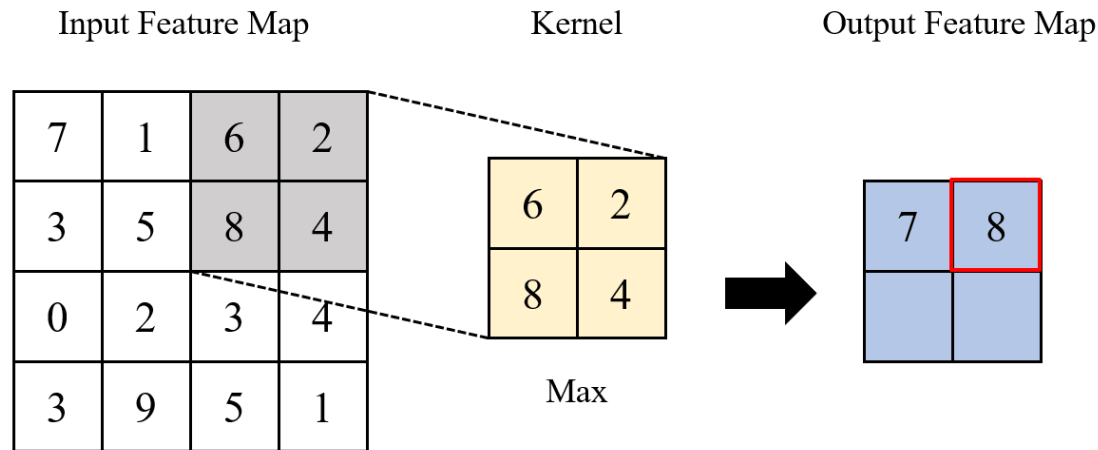


圖 2.14 池化層運算範例二

與卷積層中的卷積核操作相同，當池化核無法再往右邊移動時，會回到最左邊並往下一個步長繼續進行運算，得到輸出結果為 9，如圖 2.15 所示。

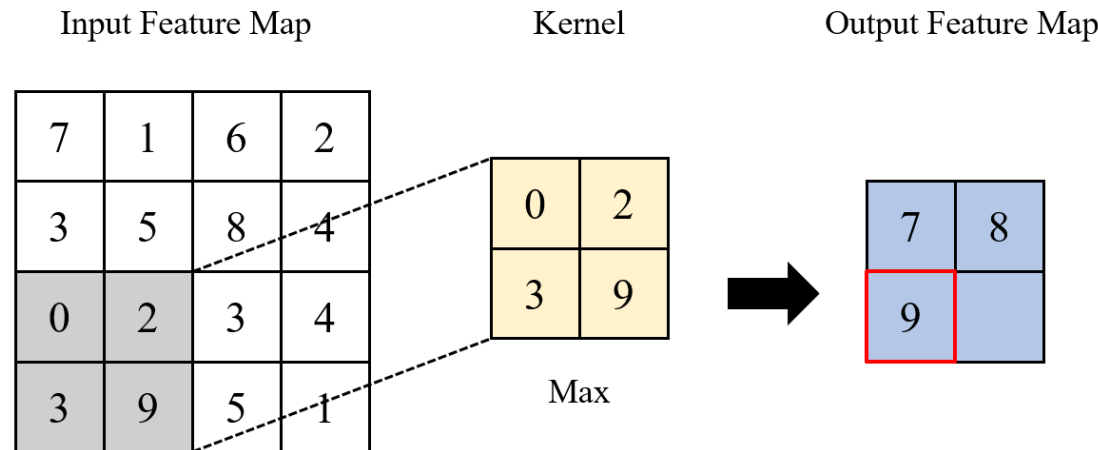


圖 2.15 池化層運算範例三

依此類推，當池化核移動到最右邊且無法再往下移動時，即完成所有的最大池化，如圖 2.16 所示，可以發現經過的池化核的運算後，輸出特徵圖的尺寸大幅縮小，並保留了重要的特徵值，當池化層結束後再將輸出特徵圖的結果傳到下一

層做運算。

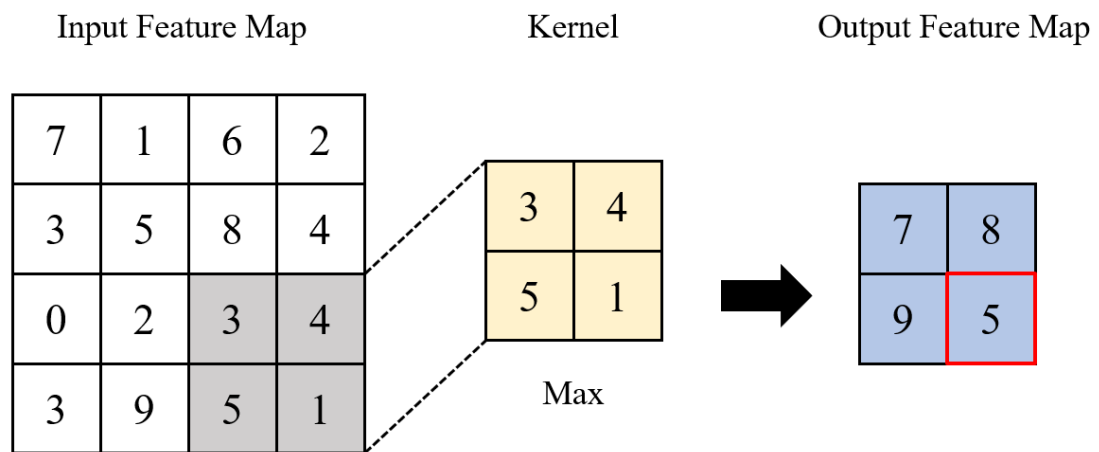


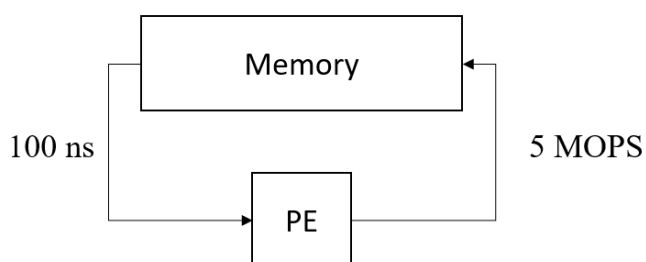
圖 2.16 池化層運算範例四

第三章 張量處理器

3.1 張量處理器之原理

張量處理器是一種專門用於加速機器學習的硬體加速器，主要用於執行深度學習的任務 [26]。張量處理器採用了高度平行與流水線 (Pipeline) 的架構，平行處理能夠同時處理多個張量操作，而流水線的架構將任務分成多個階段，將能使系統更加快速，如圖 3.1 所示，假設系統原本只有一個處理單元，且每秒可以執行五百萬個操作次數 (5 Million Operations Per Second, 5MOPS)，而張量處理器中脈動陣列的處理單元有 6 倍，且每個階段都有獨立的處理單元負責執行，則每秒可以執行的操作數將變成原本的 6 倍，從而提高整體資料的吞吐量 (Throughput)。在圖像辨識或文字辨識這種精確度要求低的任務裡，由於任務目標是找出圖片的相似性，所以對於一些輕微的錯誤是可以容忍的，因此張量處理器在進行深度學習計算時通常使用低精確度 (8 位元浮點數)，這種低精確度的計算可以加速訓練和推理過程，同時減少儲存需求和頻寬 (Bandwidth) 要求。

Instead of :



We have:

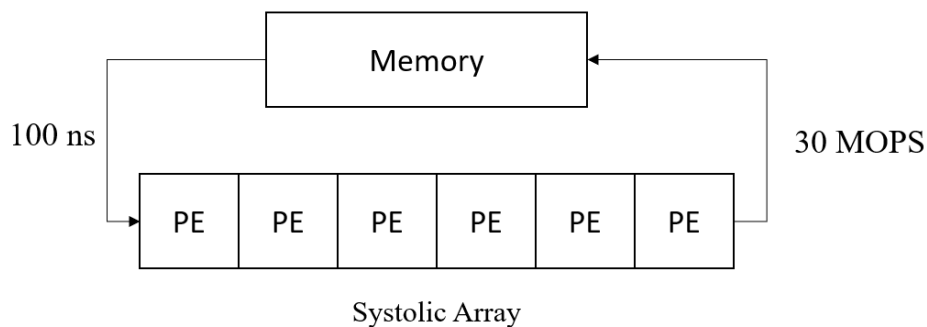


圖 3.1 張量處理器之運算原理示意圖

3.2 張量處理器之架構

張量處理器架構最主要的元件為矩陣乘法器，用於執行深度學習中常見的矩陣乘法運算，將兩個矩陣做乘積並相加後形成一個新的矩陣，而矩陣乘法器是由大量的處理單元所組成 [27,28]，如圖 3.2 所示，此架構的輸入精確度為 8 位元，輸入進來的權重值與激活值分別會存進權重暫存器與激活暫存器中，而每個處理單元裡的元件主要由一個乘法器與一個加法器所組成，將暫存器裡的權重值與激活值進行乘法運算，再與上一級處理單元的部分和做相加，並將計算完的結果存進部分和的暫存器裡，之後再往下一級處理單元進行傳遞，以此方式來達到矩陣乘積並累加的計算。

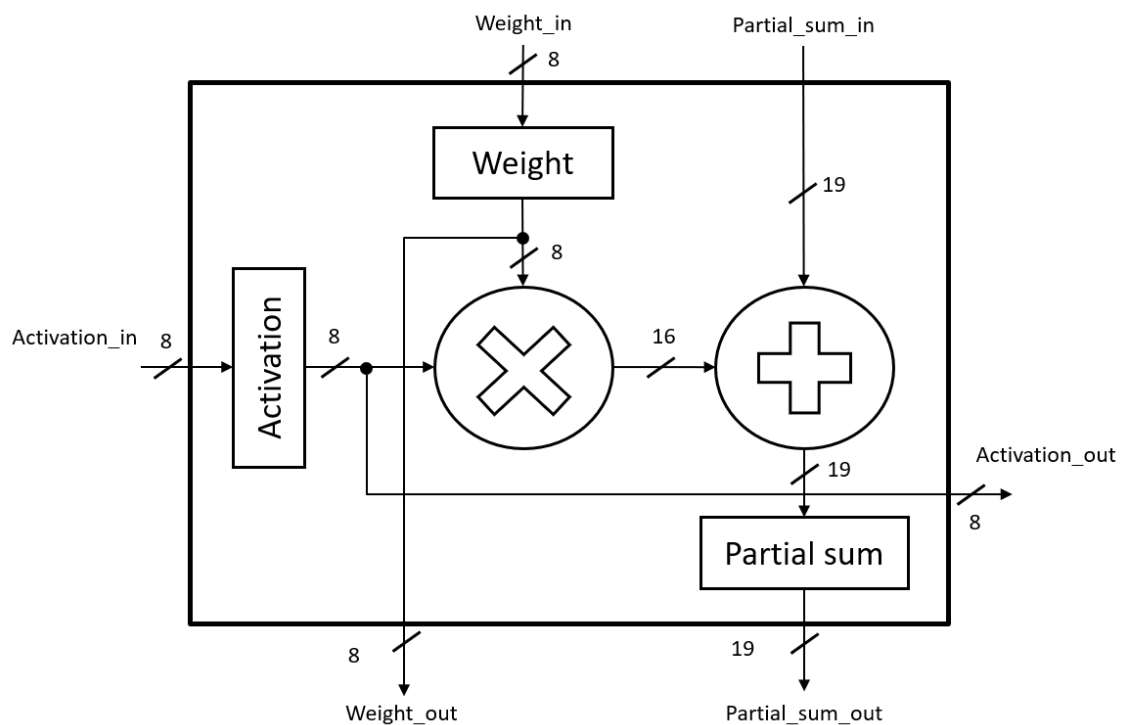


圖 3.2 單一個處理單元之示意圖

如圖 3.3 所示，矩陣乘法器的架構是將眾多的處理單元相互連接所組成的脈動陣列，張量處理器實現了高度的平行性，使其能夠在同一個時間點將大量的資料引入脈動陣列進行運算，以此來提升計算頻寬，同時脈動陣列也具有流水線架構，能夠在同一個時間點進行多個乘法與加法的操作，進而提高整體的計算速度。

進行推理運算需要執行頻繁的資料讀取操作，此讀取操作造成大量的功率消耗，因為需要針對記憶體進行存取，而硬體加速器針對此問題進行了改善。脈動陣列的上方與左方分別是權重記憶體 (Weight Memory) 與激活記憶體 (Activation Memory)，當權重值從權重記憶體讀出來時，會在脈動陣列中的處理單元內停留一段時間，以便在執行時間內重複的使用，等到權重值與激活值運算完成後，再從權重記憶體引入新的一組權重值，如此一來便可以大幅地減少權重記憶體的存取，以節省功率的消耗，而處理單元之間緊密地相連著，使資料傳遞距離縮短，不僅可以節省空間的使用，同時提高傳輸速度並減少傳輸能量，達到更進一步的功耗減少。

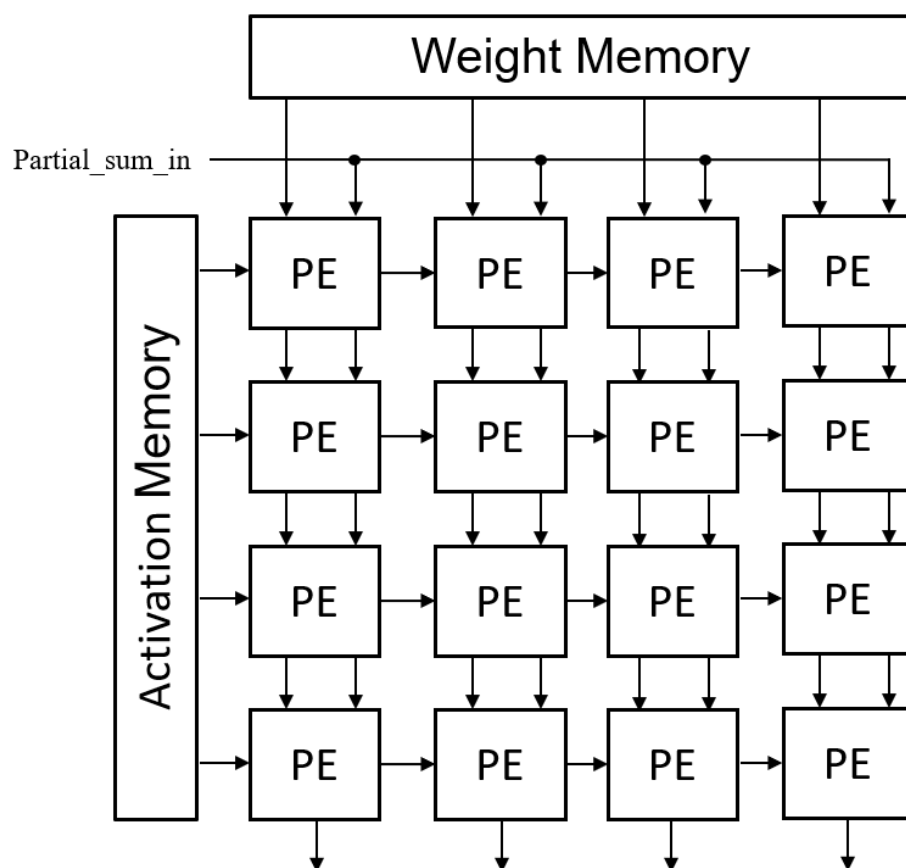


圖 3.3 矩陣乘法器之架構示意圖

脈動陣列的運算結果會暫時存進脈動陣列底部的累加器 (Accumulator) 裡，每當換一組權重值後，其輸出結果會與原先的輸出結果累加，直到某一個神經元

的所有權重值與激活值皆計算完成後，累加器會將最終的計算結果輸出到激活函數做非線性的轉換，再經過標準化 (Normalize) 來緩解梯度消失的問題，並加速訓練的收斂程度，然後進行池化 (Pooling) 來縮減資料的尺寸大小，以減少計算的複雜度，最後將處理完成後的資料全部儲存在統一緩衝器 (Unified Buffer)，作為下一層神經網路的輸入，如圖 3.4 所示。

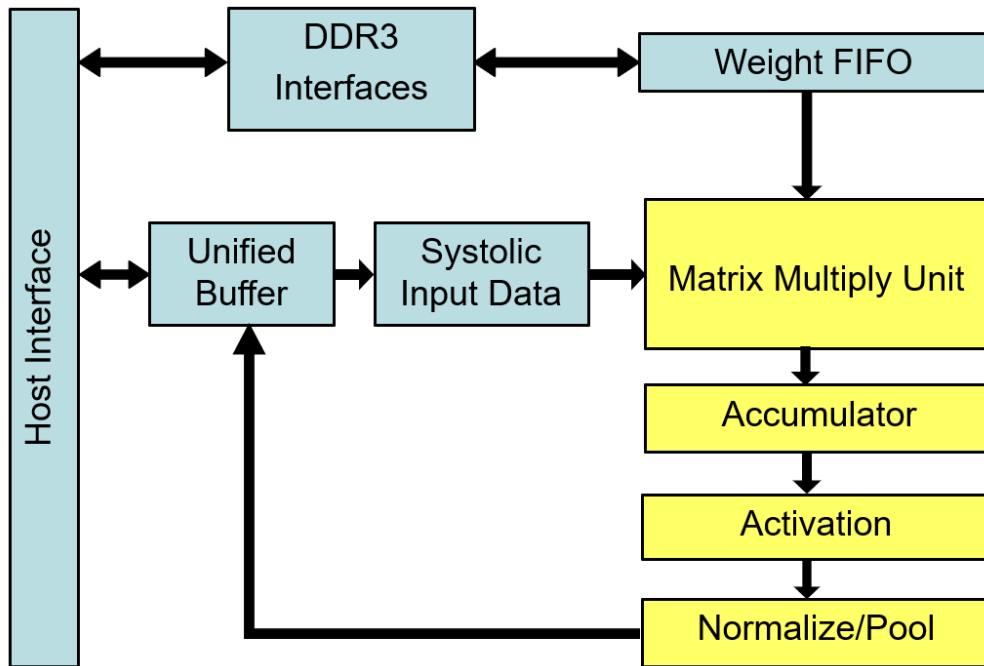


圖 3.4 張量處理器之架構示意圖 [1]

3.3 張量處理器之操作流程

在深度神經網路中，常用的計算包括卷積運算與矩陣乘法，張量處理器進行卷積運算的方式主要是依靠脈動陣列的架構來實現，首先將二維的激活矩陣轉換成一維的資料陣列，以便將資料送入脈動陣列做計算，範例以 X 矩陣表示激活矩陣，再來把權重矩陣當作卷積核，將卷積核每一步可以對應到的所有激活矩陣轉換成一維輸入陣列的形式，便可以送入脈動陣列進行卷積運算，範例以 W 矩陣表示權重矩陣。

矩陣乘法運算是將二維的兩個輸入矩陣同時送入脈動陣列，每個時脈

(Clock) 會將輸入的資料往下一個處理單元進行傳遞，而當前計算完的結果會暫時儲存於當前的處理單元做後續的累加，最後輸出的結果將會儲存在各個處理單元的暫存器內部。接下來將介紹兩種脈動陣列的運算形式，分別是權重固定 (Weight Stationary) 與輸出固定 (Output Stationary) [29, 30]。

3.3.1 權重固定之資料流

張量處理器若要執行卷積運算，首先要將二維的輸入矩陣轉換成一維的資料陣列，以便將資料送入脈動陣列做計算，如圖 3.5 所示，輸入為 3×3 的二維矩陣，我們藉由 2×2 的卷積核做運算，範例的步長為 1，將卷積核每一步可以對應到的所有 2×2 輸入矩陣轉換成一維輸入陣列的形式，如此一來便可以送入脈動陣列進行卷積運算。

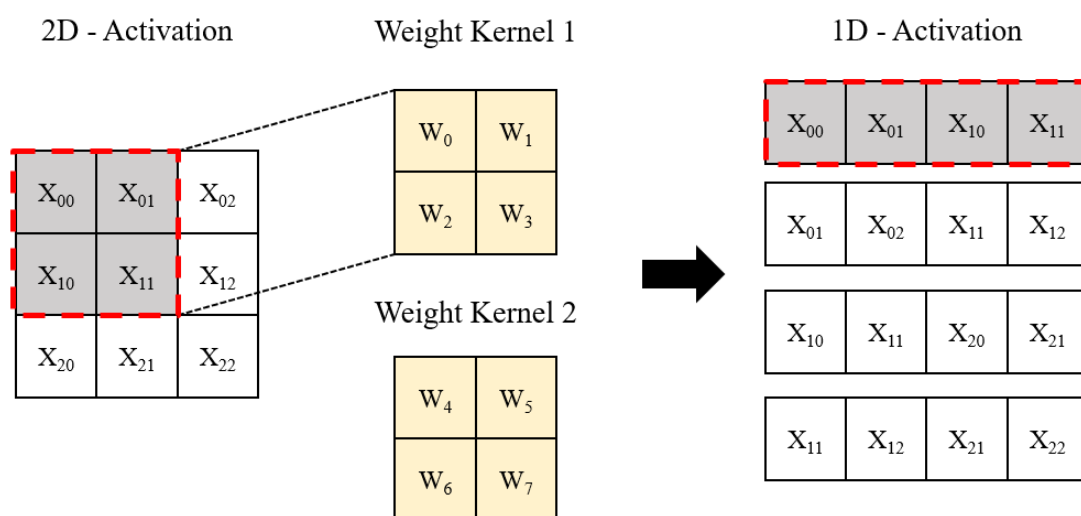


圖 3.5 二維輸入矩陣轉換成一維輸入陣列之示意圖

以 4×2 的脈動陣列為例，如圖 3.6 所示，在權重固定的條件下，首先脈動陣列會事先從權重記憶體中讀取出權重值，為了在計算過程中重複的利用權重值，輸入進來的權重矩陣會在處理單元的暫存器內維持一段時間，接著激活值會以 45 度的輸入方式送進脈動陣列裡。

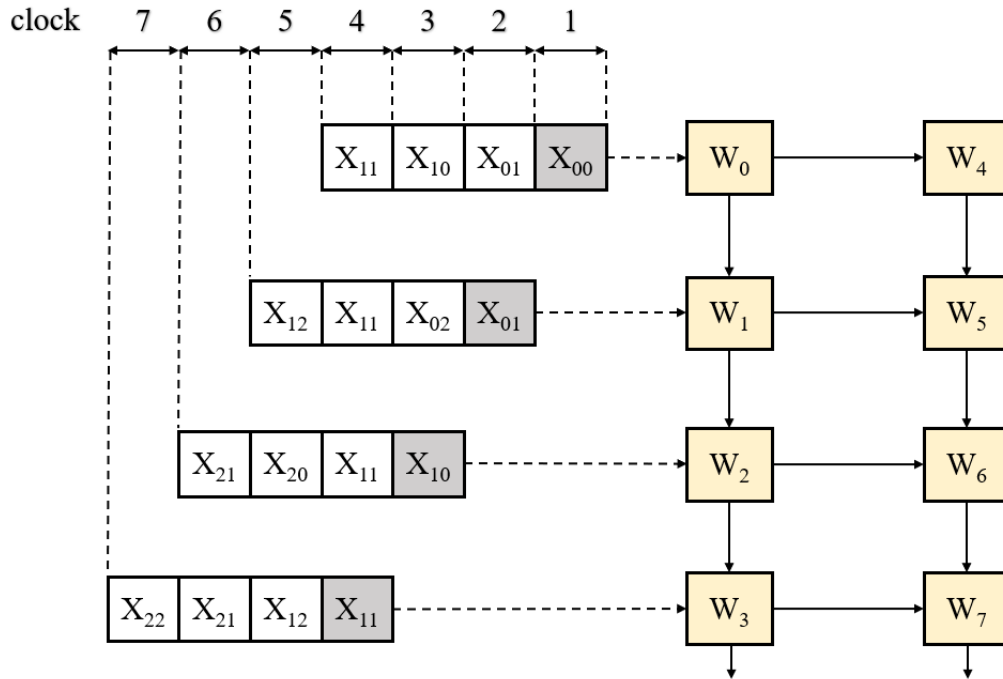


圖 3.6 脈動陣列之卷積運算範例一

當第一個時脈觸發時，激活矩陣的第一個值 X_{00} 會與 W_0 做相乘，相乘的結果會往下傳遞給下方的處理單元做後續的累加，而 X_{00} 則向右傳遞，作為右方處理單元的輸入激活值，如圖 3.7 所示。

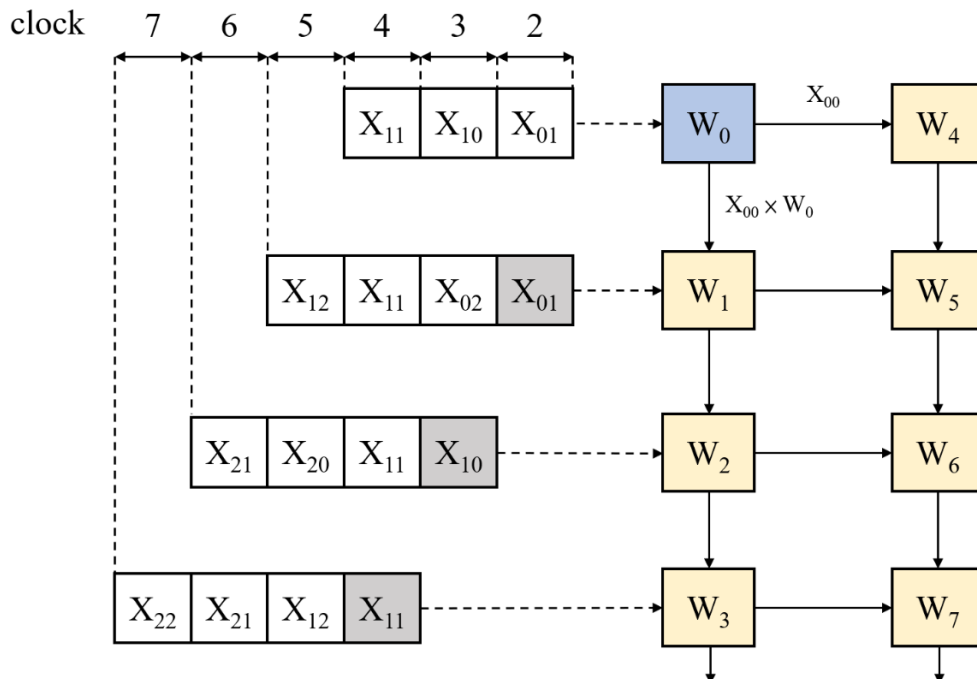


圖 3.7 脈動陣列之卷積運算範例二

第二個時脈觸發時，兩個激活值 X_{01} 輸入進來分別與 W_0 與 W_1 做相乘，相乘後的 $X_{01} \times W_1$ 與上一個時脈的 $X_{00} \times W_0$ 做累加，並繼續往下傳遞，而相乘後的 $X_{01} \times W_0$ 則是取代上一個時脈的 $X_{00} \times W_0$ ，準備與下一個時脈的相乘結果做累加。而在第一個時脈向右傳遞的激活值 X_{00} ，此時則與第二行 (Column) 的 W_4 相乘，並繼續從第二行往下傳遞，如圖 3.8 所示。

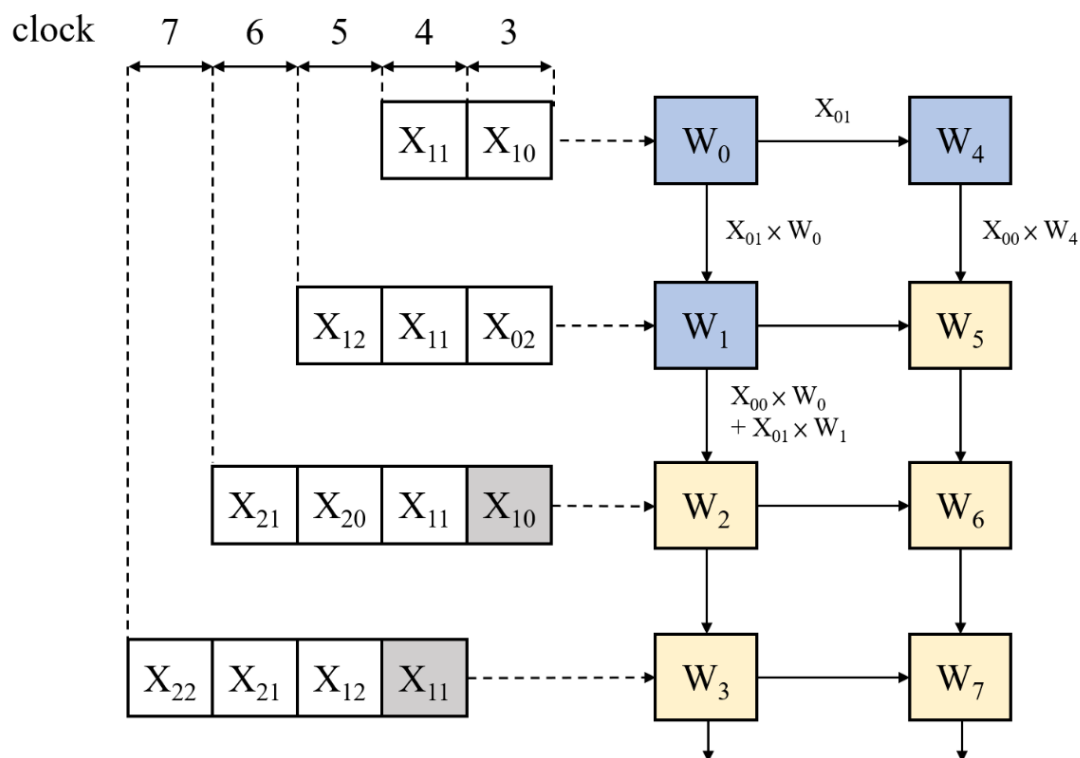


圖 3.8 脈動陣列之卷積運算範例三

當第四個時脈觸發時，可以看到第一個卷積結果已經從第一行且最後一列 (Row) 的處理單元傳遞出去，而其他的處理單元也陸陸續續的算出其他卷積的一部分結果，由於使用流水線的架構，接下來每個時脈觸發時，都可以輸出一個甚至多個卷積結果，整體的吞吐量隨著脈動陣列的大小而有所改變，如圖 3.9 所示。

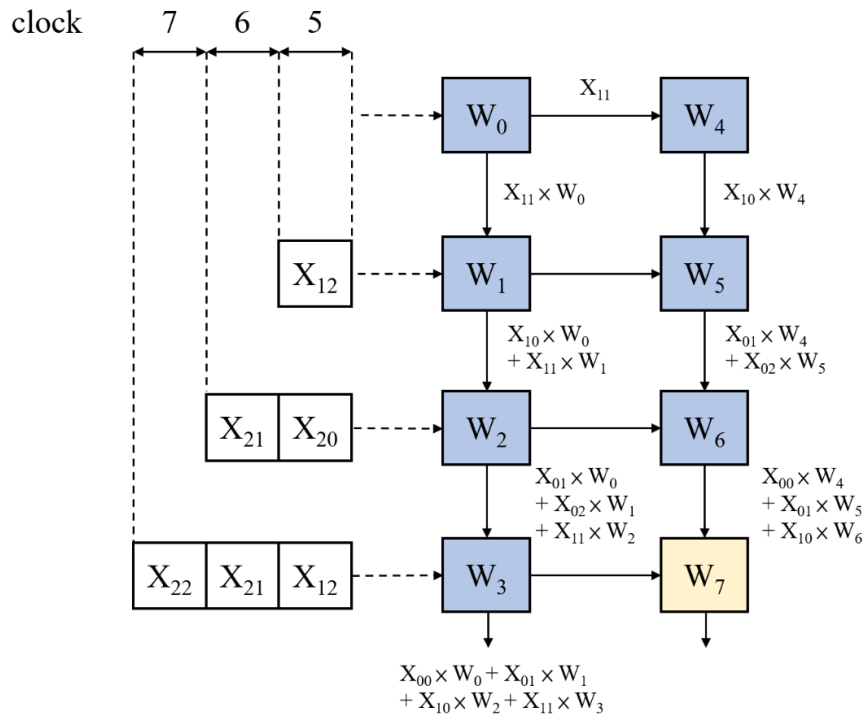


圖 3.9 脈動陣列之卷積運算範例四

直到所有的時脈觸發完畢後，可以看到最後一個卷積結果從最後一行且最後一列的處理單元傳遞出去，而其他的處理單元也都沒有輸入值可以繼續進行計算，即為計算完成，如圖 3.10 所示。

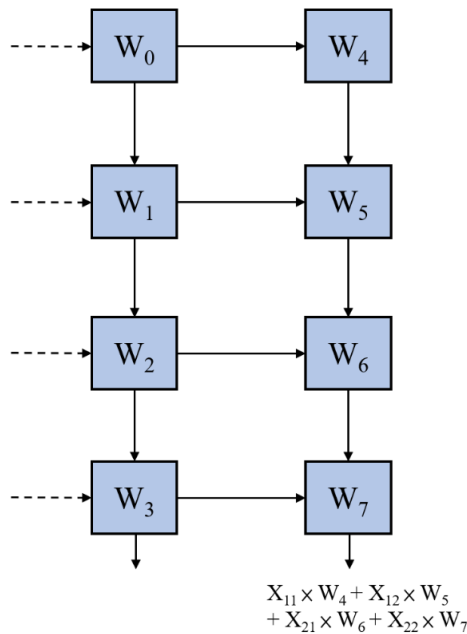


圖 3.10 脈動陣列之卷積運算範例五

3.3.2 輸出固定之資料流

如圖 3.11 為 X 激活矩陣與 W 權重矩陣的矩陣相乘獲得 Y 輸出矩陣的示意圖。如圖 3.12 所示，張量處理器若要執行矩陣運算，首先要將兩個矩陣的資料分別從脈動陣列的左方與上方同時以 45 度的傳輸方式送入，脈動陣列會從左上角的處理單元開始計算，每當時間週期觸發後，矩陣資料會分別往右方與下方傳遞，並將計算後的結果存進處理單元的暫存器中。

$$\begin{pmatrix} X_{00} & X_{01} & X_{02} \\ X_{10} & X_{11} & X_{12} \\ X_{20} & X_{21} & X_{22} \end{pmatrix} \begin{pmatrix} W_{00} & W_{01} & W_{02} \\ W_{10} & W_{11} & W_{12} \\ W_{20} & W_{21} & W_{22} \end{pmatrix} = \begin{pmatrix} Y_{00} & Y_{01} & Y_{02} \\ Y_{10} & Y_{11} & Y_{12} \\ Y_{20} & Y_{21} & Y_{22} \end{pmatrix}$$

圖 3.11 矩陣運算之示意圖

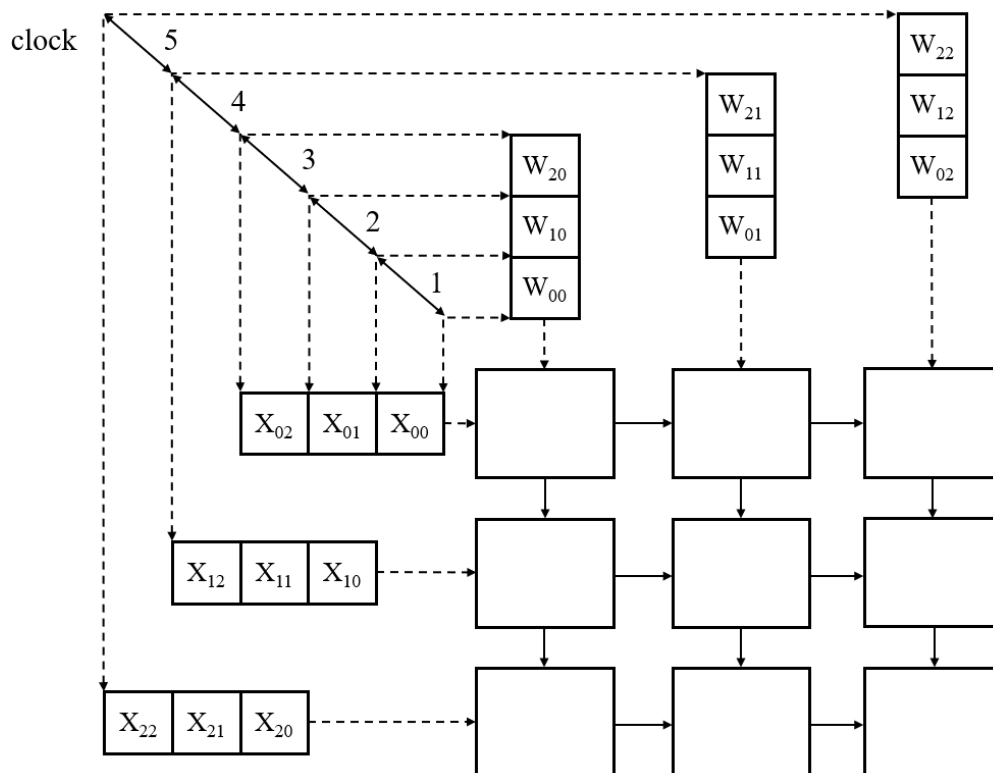


圖 3.12 脈動陣列之矩陣運算範例一

在第一個時脈觸發時，激活值 X_{00} 與權重值 W_{00} 送入左上角的第一個處理單元進行相乘，並將相乘後的結果儲存在當前的處理單元內，而激活值 X_{00} 與權重值 W_{00} 分別往右方與下方的處理單元傳遞，當作下一級的輸入，準備與下一級的資料做乘法運算，如圖 3.13 所示。

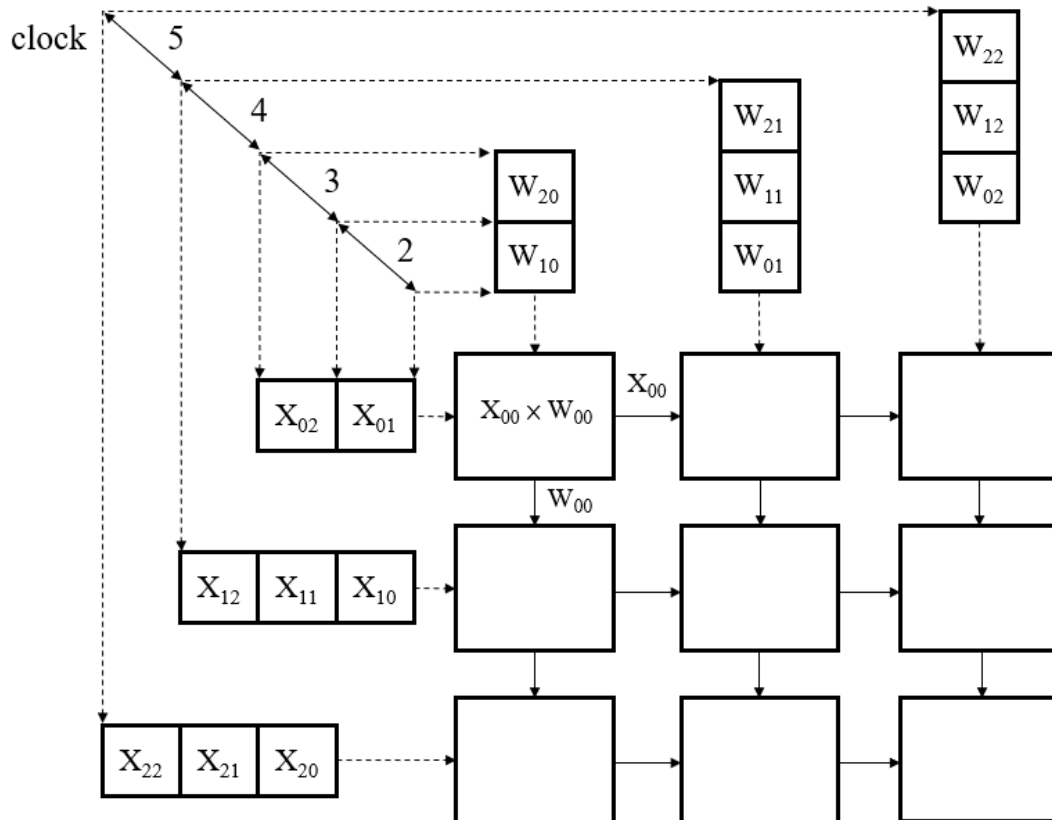


圖 3.13 脈動陣列之矩陣運算範例二

在第二個時脈觸發時，激活值 X_{01} 與權重值 W_{10} 送入左上角的第一個處理單元進行相乘，並與先前的 $X_{00} \times W_{00}$ 做相加並繼續儲存於暫存器內，而上一個時脈的激活值 X_{00} 與權重值 W_{00} 分別與權重值 W_{01} 和激活值 X_{10} 相乘並儲存於當前的處理單元中，而激活值 X_{01} 與權重值 W_{10} 與先前的激活值 X_{00} 與權重值 W_{00} 一樣繼續往右方與下方的處理單元傳遞，當作下一級的輸入，繼續準備與下一級的資料做乘法運算，如圖 3.14 所示。

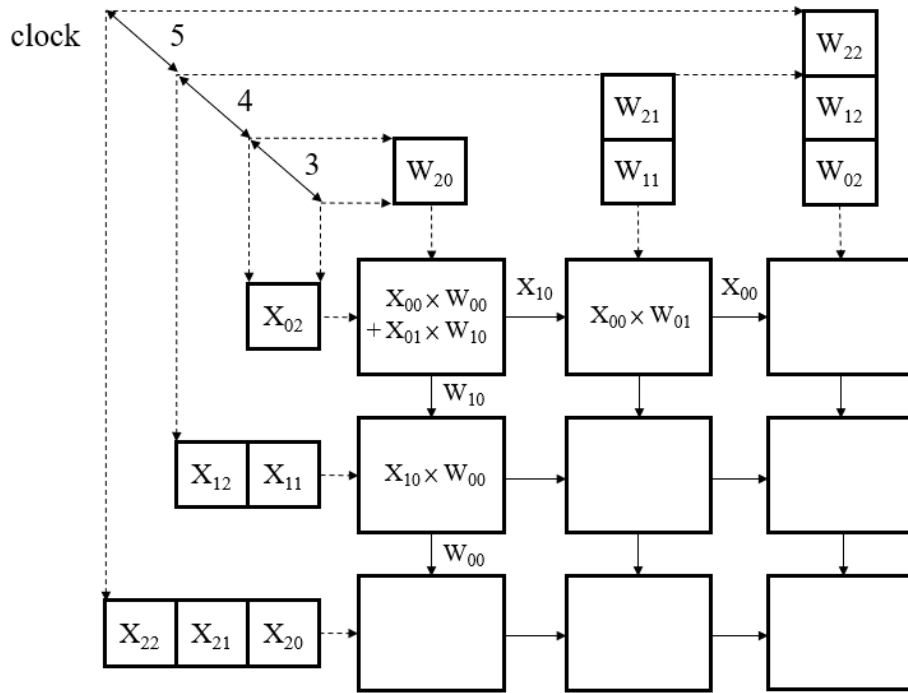


圖 3.14 脈動陣列之矩陣運算範例三

當所有時脈觸發完後，也就代表矩陣乘法的運算已執行完畢，可以看到激活矩陣 X 與權重矩陣 W 的相乘並累加的結果皆已存儲在各個處理單元內部的暫存器中，此結果即為輸出矩陣 Y ，如圖 3.15 所示。

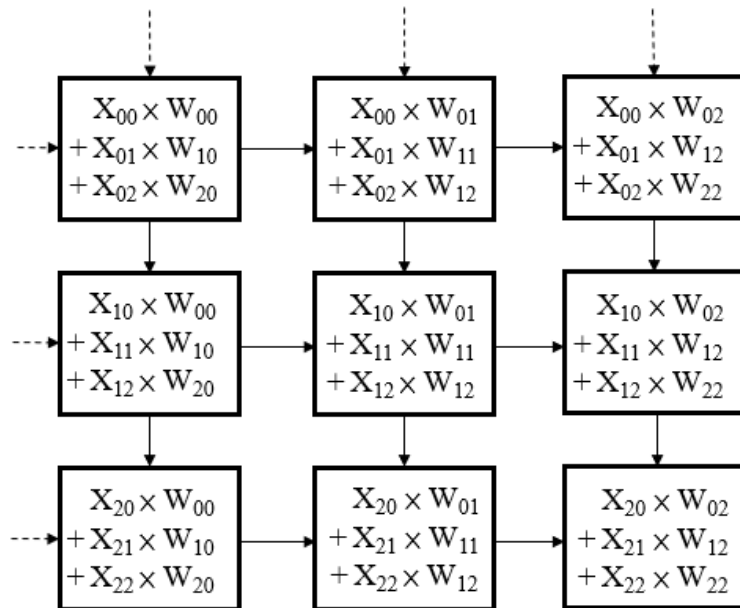


圖 3.15 脈動陣列之矩陣運算範例四

3.4 降低成本之張量處理器之相關研究

3.4.1 動態範圍之無偏乘法器 (Dynamic Range Unbiased Multiplier, DRUM) [2]

在低成本乘法器的設計中，此研究提出了動態的近似計算來縮減乘法器的大小，且同時引入無偏 (Unbiased) 誤差來確保乘法器計算結果的準確度。在乘法計算的過程中，假設每個資料有 n 個位元，我們將設計兩個領頭一偵測器 (Leading One Detector, LOD)，用來偵測無號 (Unsigned) 數在哪個位置出現了第一個有效位元，並將其前面高位元為 0 的所有位元資料做捨棄，因為它們不會影響計算結果，接著根據使用者的準確度需求，從無號數第一個有效位元的位置開始往後取 k 個位元的精確度來當作乘法器的輸入，而在第 k 個之後的位元無條件捨去，我們計算捨棄掉的這些位元它們的期望值，假設我們捨棄了 x 個位元，且每個位元出現的機率都一樣，那他們的期望值剛好就會是 2^{x-1} ，也就是如果我們捨棄了 4 個位元，期望值將會是 4 位元的 1000，所以我們將第 k 個位元設定為 1 當作期望值來取代那些被捨棄的值，以確保進一步的資料長度縮短，但是需要藉由捨棄掉的位元個數來計算位移次數，把後續乘法器的計算結果移位回原本的近似值，如圖 3.16 所示。

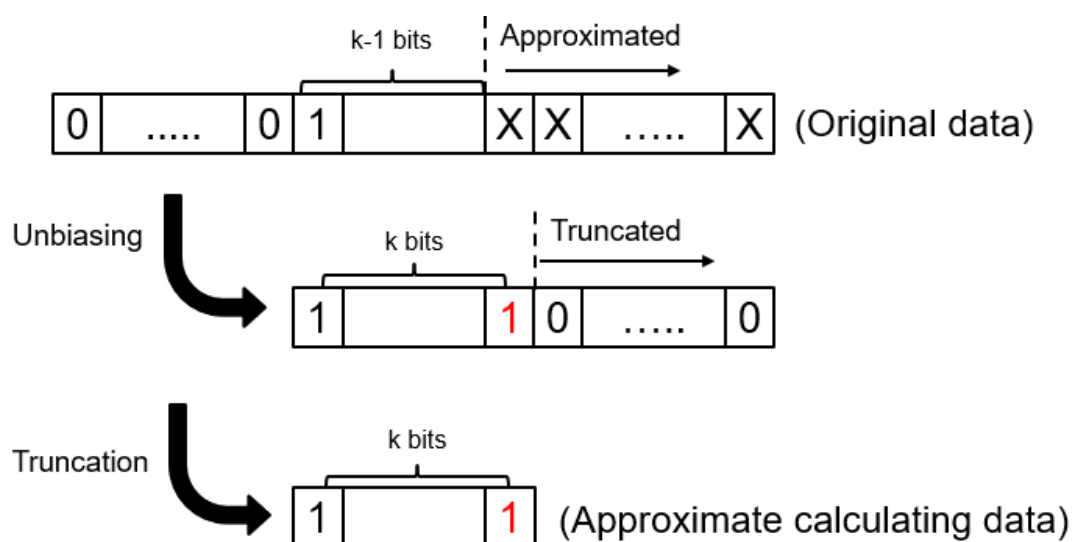


圖 3.16 縮短資料字長之示意圖 [2]

圖 3.17 為無偏乘法器執行乘法計算的範例，第一步先輸入兩個乘法計算的資料，第二步則分別找出兩個資料的第一個有效位元並捨棄其前面所有的位元資料，接著從有效位元開始擷取 k 個位元的精確度，範例精確度為 6 個位元，將第 6 個位置的位元設為 1 並將未被擷取到的位元捨去，並分別記錄 7 個位移次數與 3 個位移次數，第三步將兩個精確度為 6 位元的資料相乘獲得了一個計算結果，最後將兩個資料的位移次數相加獲得 10 個位移次數，再將此計算結果向左移位 10 次，得到一個近似的計算結果，如圖 3.17 的第四點所示，可以看到其近似結果與圖 3.17 的第五點的正確結果相差不多，誤差的大小則依據使用者所能接受的範圍而定。

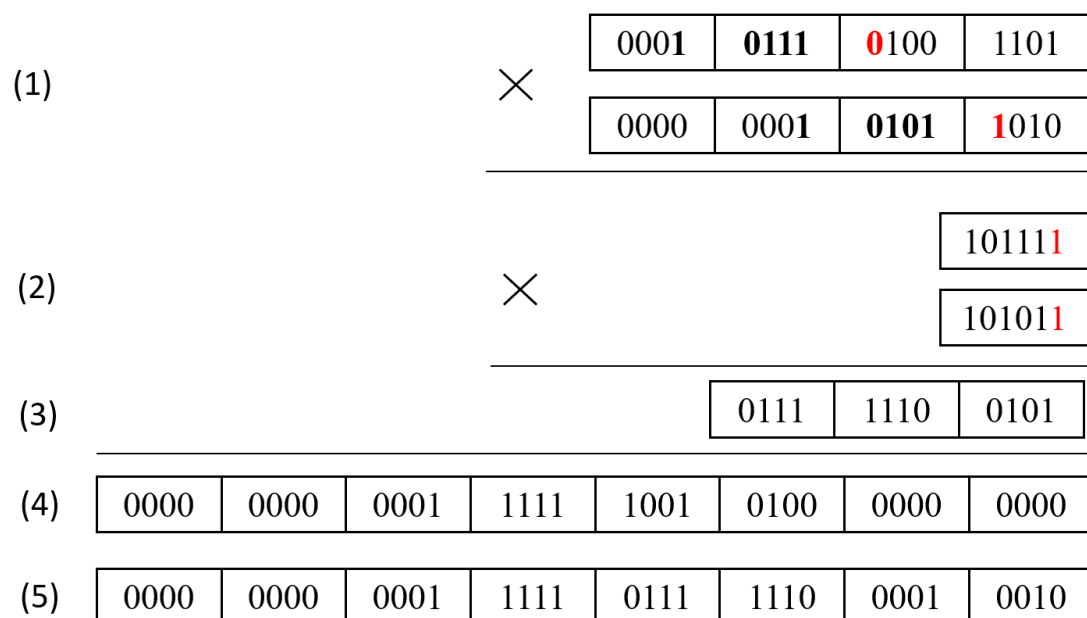


圖 3.17 無偏乘法器之乘法計算示意圖 [2]

圖 3.18 為無偏乘法器的硬體架構，圖中 A 與 B 為輸入乘法器的兩個資料，經過領頭一偵測器發現第一個有效位元的位置，然後經過編碼器 (Encoder) 來計算出所需要的位移次數與多工器 (Multiplexer, MUX) 要選擇多少位元的精確度，接著將多工器選出的資料經過乘法器做相乘，同時把兩個資料的位移次數透過加法器做相加，最後桶式位移器 (Barrel Shifter) 會將乘法的計算結果向左進行移位，移位的次數為兩者位移次數的總和，移位後的輸出結果即為無偏乘法器計算

出的乘法近似值。

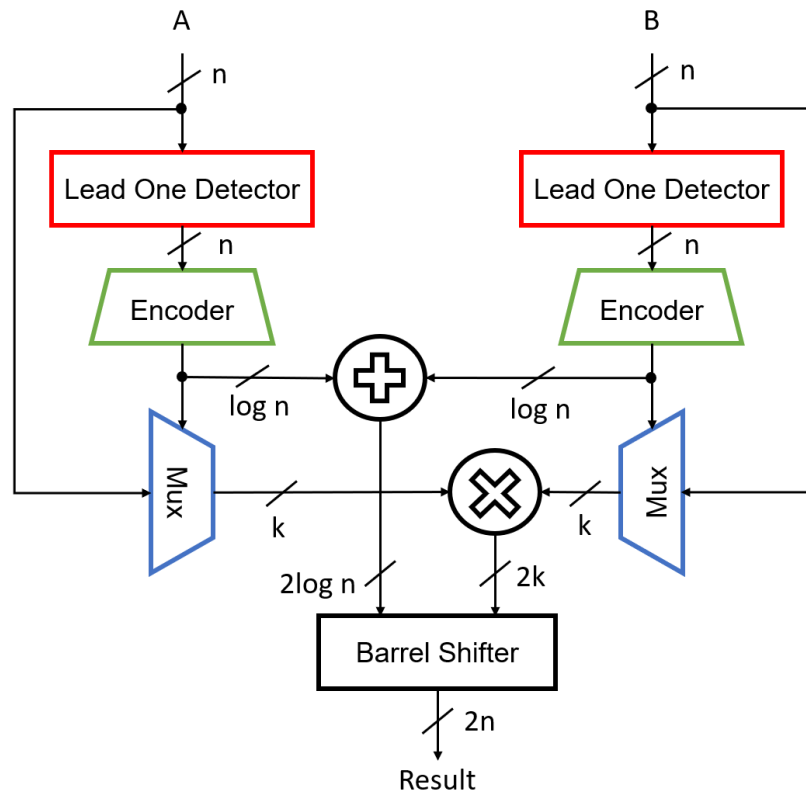


圖 3.18 無偏乘法器之硬體架構 [2]

3.4.2 近似計算之張量處理器 (Approximate Tensor Processing Unit, APTPU) [3]

近似計算的張量處理器延用動態範圍的無偏乘法器的概念，將張量處理器中的處理單元內部的乘法器調整為近似乘法器，為了不使準確度大幅降低，同時減少硬體的成本。如圖 3.19 所示，處理器中使用預近似單元 (Pre-Approximate Unit, PAU) 來縮短輸入資料的精確度與紀錄位移的次數，利用領頭一偵測器來找出第一個有效位元的位置，並擷取使用者所需要的資料精確度，最後藉由比較器與減法運算來計算出所需的位移次數，同時記錄符號位元來讓後續的處理單元可以執行正數與負數的運算。

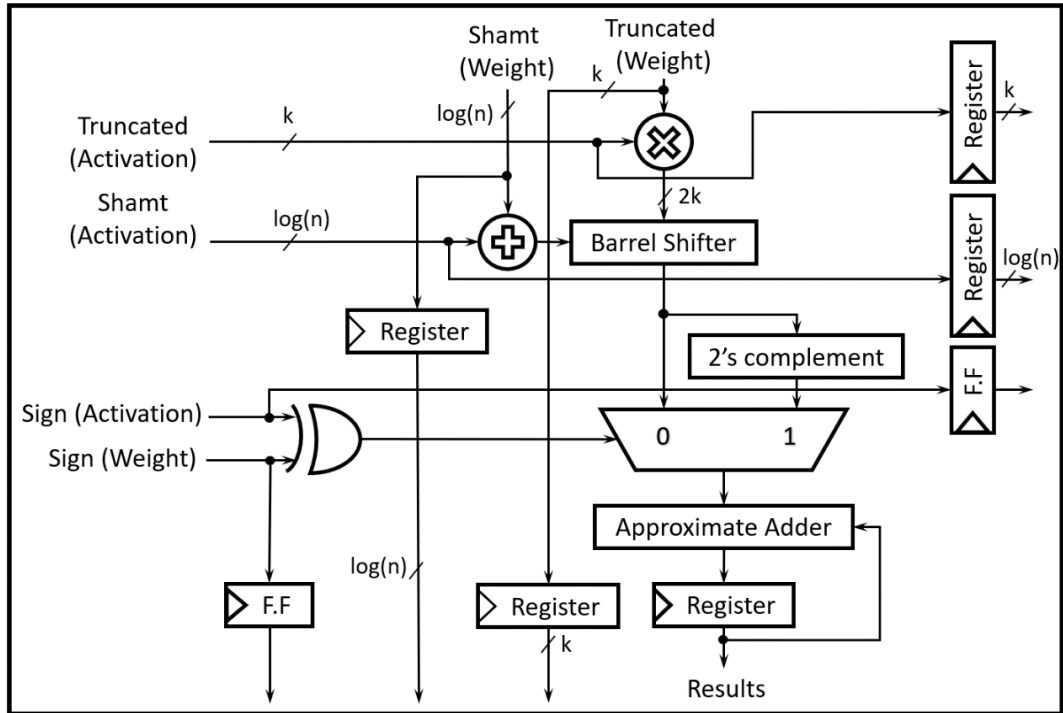


圖 3.20 近似處理單元之硬體架構 [3]

如圖 3.21 所示，為近似張量處理器整體的架構，將近似處理單元之間互相連接來組成一個脈動陣列，脈動陣列與記憶體的緩衝區 (Buffer) 之間分別設置一個預近似單元來縮短輸入資料的長度，再藉由控制器 (Controller) 來控制輸入的資料流，最後在做深度學習的卷積運算或矩陣運算時，將脈動陣列的輸出結果存進輸出記憶體 (Output Memory) 中，如此一來就形成了低成本的近似張量處理器。

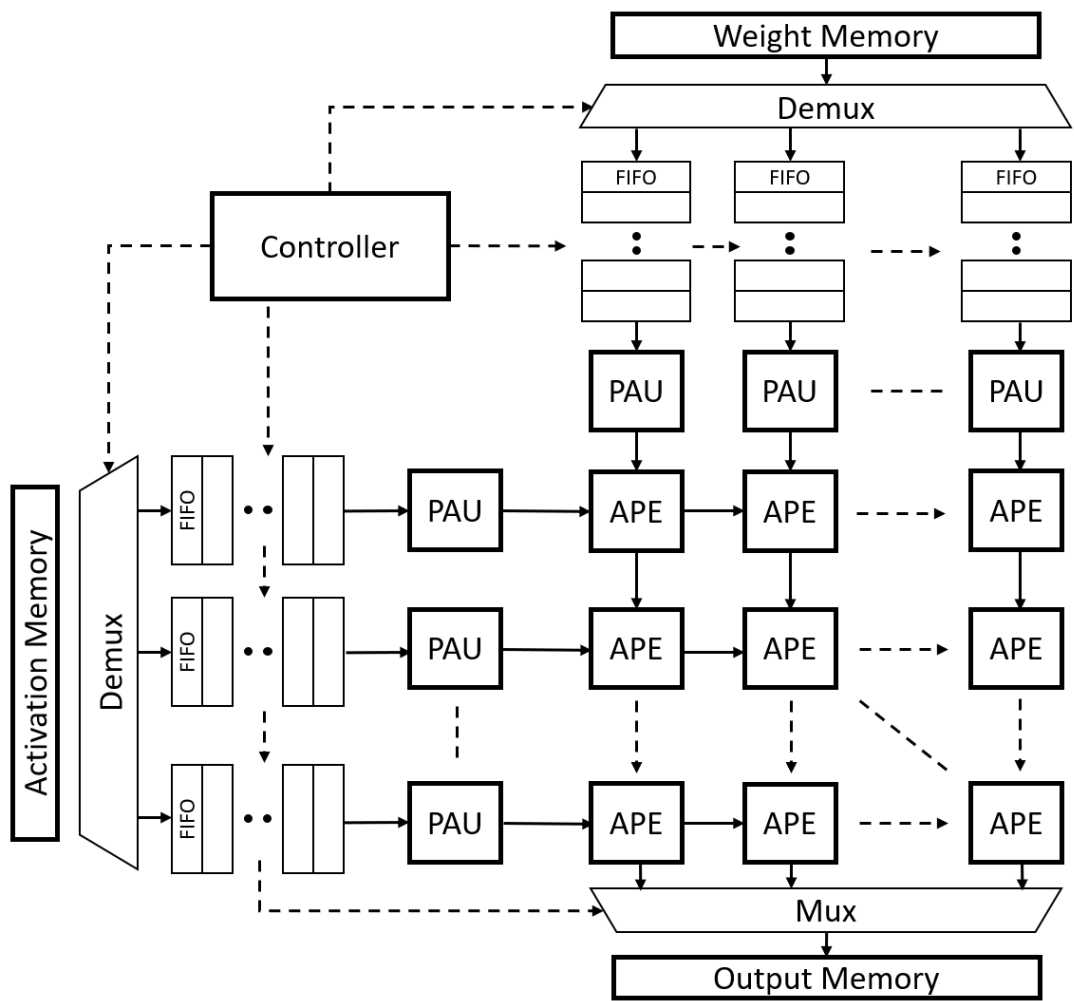


圖 3.21 近似張量處理器之硬體架構 [3]

第四章 降低位元精確度之張量處理器

4.1 權重值分佈之觀察

通常深度神經網絡模型使用 32 位元浮點數 (Floating Point) 運算進行訓練。訓練完成後可以獲得 32 位元的權重值。然而，為了減少計算資源和時間，深度神經網路通常使用定點數運算進行推論計算 [31]。由於大多數權重的值接近於零，我們可以觀察所有權重值中重要的連續位元分佈狀況，如圖 4.1 所示，我們將權重值以定點數格式轉換為 8 位元的二進制資料並觀察分佈的情況，橫坐標為權重值含有 n 個重要的連續位元，縱座標為 n 個重要連續位元的權重值佔據所有權重數量的多少百分比。如表 4.1 所示，可以看到將近 88% 以上的權重值都含有 5 個連續的重要位元，甚至 99% 以上的權重值含有 4 個連續的重要位元，透過這次觀察，我們試著將權重值中的 4 個連續的重要位元以 1 個符號位元來代替，藉此來省下張量處理器中的硬體成本，同時觀察縮短精確度後的權重值是否對於整體的準確度有任何影響。

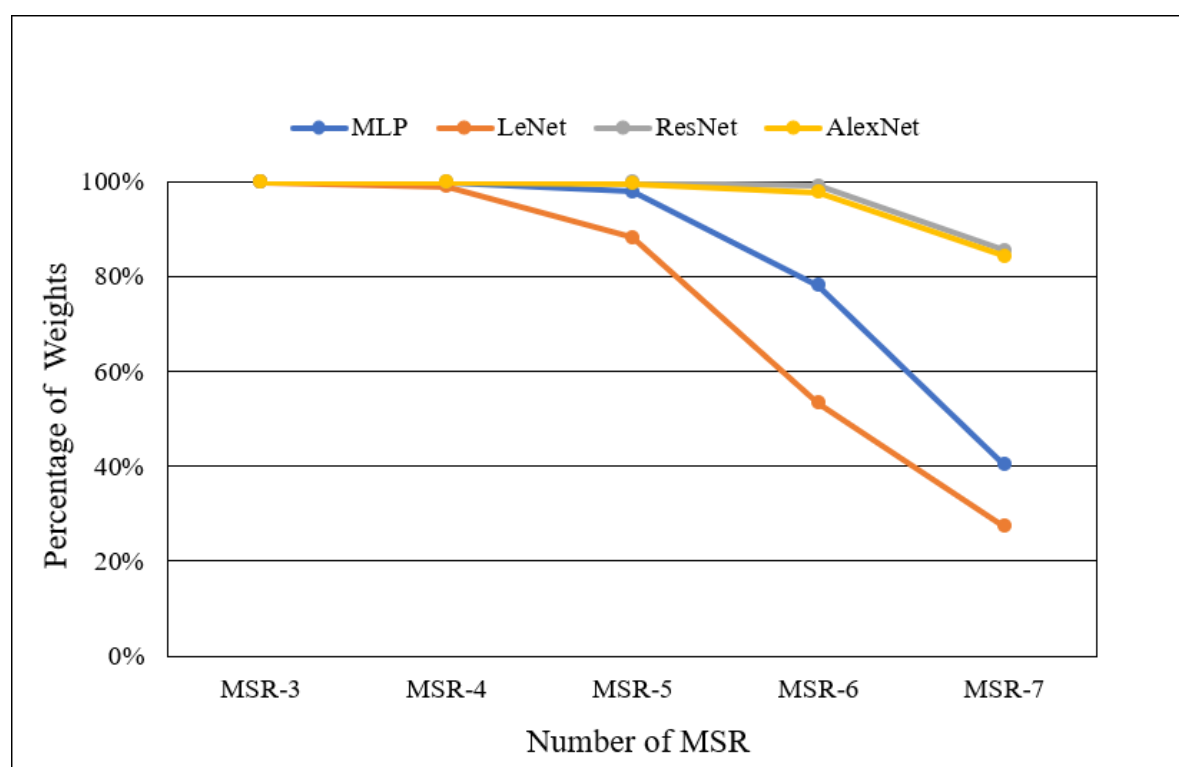


圖 4.1 權重之 MSR 分佈圖

表 4.1 權重之 MSR 分佈表

	MLP	LeNet	ResNet	AlexNet
MSR-3	99.9%	99.9%	99.9%	99.9%
MSR-4	99.8%	99.0%	99.9%	99.9%
MSR-5	98.0%	88.3%	99.9%	99.7%
MSR-6	78.2%	53.4%	99.1%	97.8%
MSR-7	40.4%	27.3%	85.5%	84.3%

4.2 權重值預處理之方式

透過觀察，我們得知絕大部分的權重值都可以進行字長的縮減，所以如果我們事先將權重值縮減後再存進記憶體，不僅可以減少記憶體的面積，還可以縮小後續張量處理器中的處理單元的硬體成本。如圖 4.2 為權重值進行字長縮減的示意圖，圖中的左方為模型已經訓練好的原始權重值，接著判斷每個權重值是否含有重要的連續位元，本篇論文以 4 個重要的連續位元來判斷，如果判斷結果為是的話，就以一個符號位元來取代這 4 個位元，然後存進記憶體時加上 1 個位移位元 (Shift Bit) 來讓後續的處理單元判斷是否要移位資料，舉例來說，圖中第一個權重值為 8 位元的 00001101，判斷結果有重要的連續位元，因此將高位元的 4 個 0 以 1 個 0 的符號位元來取代，也就是縮短成權重值為 5 位元的 01101，存進記憶體時附上 1 個 0 的位移位元來代表後續記憶體讀出來時不需要進行移位；圖中第二個權重值為 8 位元的 10101001，判斷結果沒有重要的連續位元，因此將最低的 3 個位元捨棄，也就是縮短成權重值為 5 位元的 10101，存進記憶體時附上 1 個 1 的位移位元來代表後續記憶體讀出來時需要進行 3 次向左移位（因為我們捨去了原本的 3 個位元），來讓失去準確度的資料近似為原本的權重值。



圖 4.2 權重縮減之示意圖

如圖 4.3 為權重預處理單元 (Weight Preprocessing Element, WPE) 的硬體架構，為了判斷從高位元開始算起的重要連續位元，我們可以簡單的將高位元用及閘 (AND Gate) 與或閘 (OR Gate) 連接起來，最後再用互斥或閘 (XOR Gate) 將及閘與或閘的輸出連接起來，如此一來我們便可以藉由互斥或閘的輸出來判斷權重值是否含有重要的連續位元，如判斷結果為是的話，互斥或閘的輸出則為 0，代表權重值沒有準確度的損失，因此被讀出後不需要進行移位，如圖 4.3(a) 所示；反之，如果判斷結果為否的話，互斥或閘的輸出則為 1，代表權重值有準確度的損失，被讀出後需要進行移位來讓資料近似為原本的權重值，如圖 4.3(b) 所示。舉例來說，將圖 4.3(a) 的輸入資料中最高的 4 個位元送入或閘與及閘進行判斷，兩者得到的結果皆為 0，因此互斥或閘的結果為 0，於是多工器選擇將最低的 5 個位元與 1 個 0 的位移位元傳送給輸出，代表此資料讀出後不需要進行移位；而右邊的圖 4.3(b) 的輸入資料中最高的 4 個位元送入及閘與或閘進行判斷，兩者得到的結果分別為 1 和 0，因此互斥或閘的結果為 1，多工器則選擇將最高的 5 個位元傳送給輸出，與 1 個 1 的位移位元傳送給輸出，代表此資料讀出後需要進行移位，如此一來權重值預處理就可以藉由這樣的硬體架構來實現。

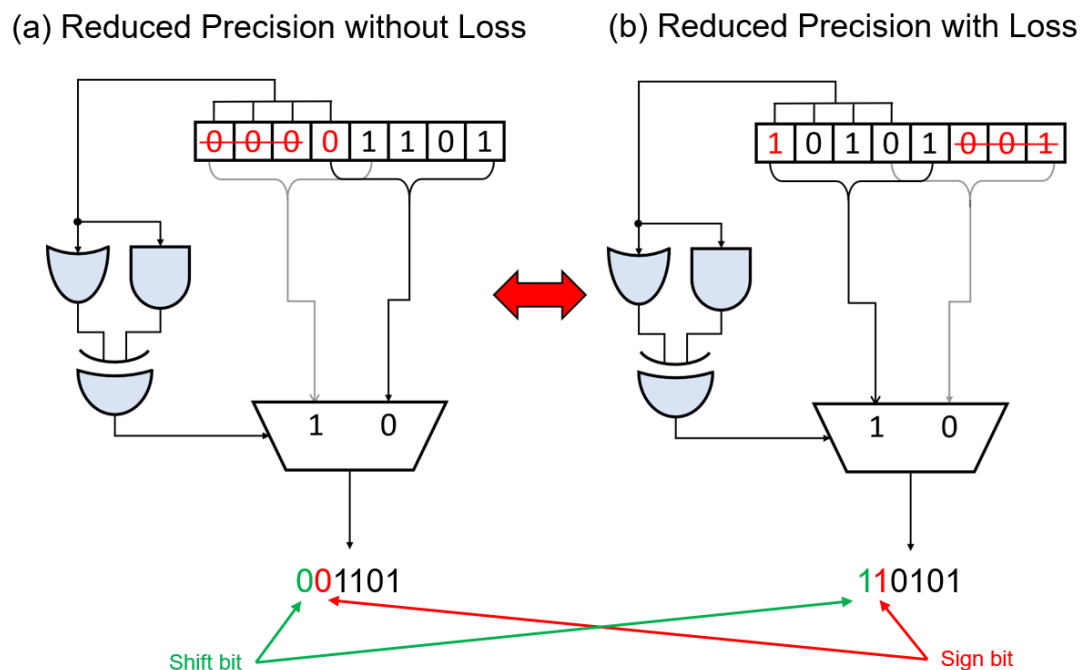


圖 4.3 權重預處理單元之架構

4.3 降低位元精確度之張量處理器原理

降低精確度之張量處理器的原理是以傳統的張量處理器為基礎而做的調整，並且針對先前的相關研究再做更進一步的提升，先前提到的近似計算的張量處理器 [3]，每當資料從記憶體讀出來時，都必須經過預近似單元來擷取特定精確度的字長，同時要記錄符號位元與位移的次數給後續的處理單元做判斷與計算，雖然這樣可以很彈性的處理不同精確度的輸入資料，但是會造成後續的處理單元需要額外的時間來計算位移次數的總和與符號位元的判斷，造成整體的效能受到限制，可能無法達到更高的操作頻率，且每個輸入資料要送進脈動陣列前都需要一個預近似單元來做預先處理的動作，隨著脈動陣列大小的增加，所需要的預近似單元硬體成本也會跟著上升。

我們為了解決近似計算的張量處理器 [3] 的計算速度與硬體成本的問題而提出了一些改良方法，既然資料每次從記憶體讀出來後都需要經過一個硬體來做精確度的處理，那我們只要事先把資料處理過後再存進記憶體，不僅可以讓記憶

體的面積縮小，同時記憶體的资料被讀出來時也就不再經過後續的處理，便可以省下多餘的硬體成本，另一方面，由於資料字長的擷取精確度不同，導致先前的近似處理單元需要額外的時間計算位移的次數，為了節省計算位移次數所花費的時間，我們只要將所有縮減後的資料字長都統一成固定的長度，如此一來紀錄下來的位移次數也將變為定值，那麼後續的處理單元只需要判斷是否進行移位即可，不要需要額外計算次數的總和。如圖 4.4 所示，為權重值送進處理單元運算的示意圖，第一種情形為權重值含有 4 個重要的連續位元，將權重值字長縮減後送進處理單元做計算，由於沒有任何準確度的損失，所以運算完的結果不需要進行移位；第二種情形為權重值不含有 4 個重要的連續位元，由於失去了 3 個位元的精確度，因此運算結果需要進行 3 次向左移位。

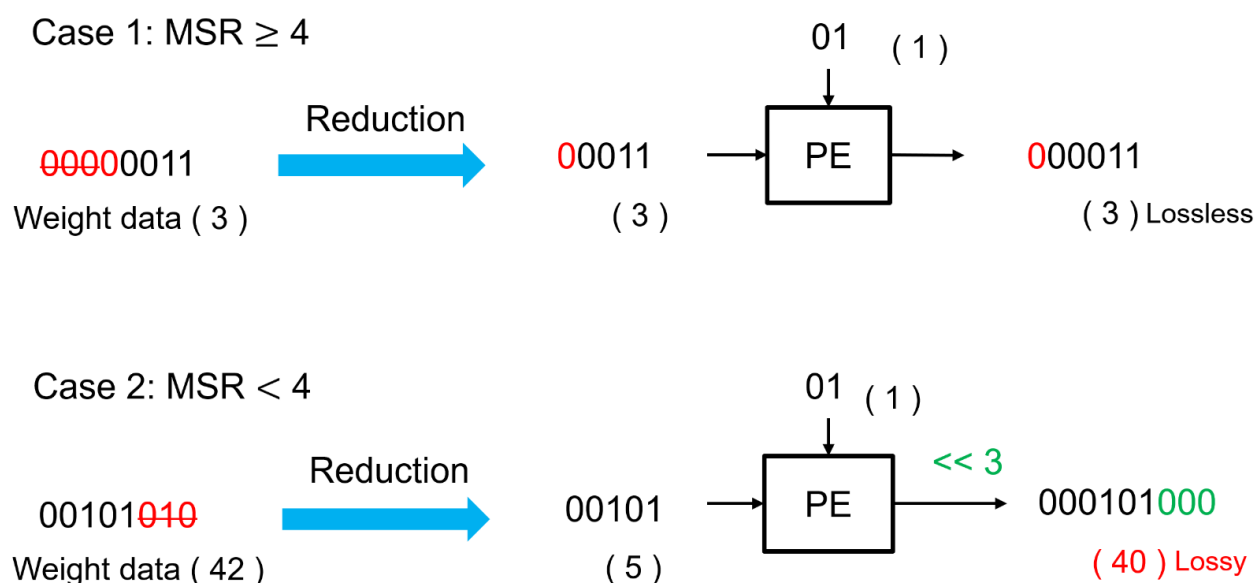


圖 4.4 權重縮減後之運算示意圖

4.4 降低位元精確度之張量處理器架構

有了權重值分佈的觀察與權重值字長縮減的處理，我們將此原理應用在張量處理器中，並針對傳統的處理單元架構進行調整，本篇論文將處理單元內存放權重值的暫存器縮減，形成降低位元精確度之處理單元 (Reduced Processing Element, RPE)，隨著權重值暫存器的面積縮小，代表著乘法器的輸入字長也會跟

著一起縮減，而字長的縮減使得乘法器內部的加法器數量減少，因此讓處理單元內部的硬體成本可以大幅的降低。本篇論文提出了降低位元精確度之張量處理器的兩種型態，並將第 i 型態的降低位元精確度之處理單元以 $RPE_i, i \in 1, 2$ 表示，第一種型態是單純的把權重值的精確度縮減所形成的架構，第二種型態則針對第一種型態準確度不足的地方進行彌補與改善，接下來的小節中將詳細介紹兩種架構的原理與內容。

4.4.1 第一型態的降低位元精確度之張量處理器架構

如圖 4.5 所示，為第一型態的降低位元精確度之處理單元，跟一般處理單元的硬體架構方面相比，我們只需多加一個多工器和位移器，由於我們將權重值輸入的字長縮減，因此用了 8×5 的乘法器，跟一般 8×8 的乘法器硬體相比大約少了八分之三。當處理單元在做計算時，將 6 位元的權重值輸入進來，然後只取最低的 5 個位元給乘法器做計算，由於我們是二補數的乘法器架構，因此符號位在乘法計算後的結果依然會得到正確的正數或負數，而最高位的位移位元將會判斷乘法結果是否需要進行移位，範例的位移位元為 0，所以不進行移位。雖然我們將 8 位元的權重值精確度縮減成 5 位元，但是考慮到某些權重值需要進行移位，最壞的情況是脈動陣列所有的權重值剛好都需要進行移位，因此輸出結果的長度依然需要使用符號擴展 (Sign Extension)，變成 16 位元才能確保資料的正確性，由於我們使用 8×8 脈動陣列的架構，因此部分和使用了 19 位元來確保資料的正確性。

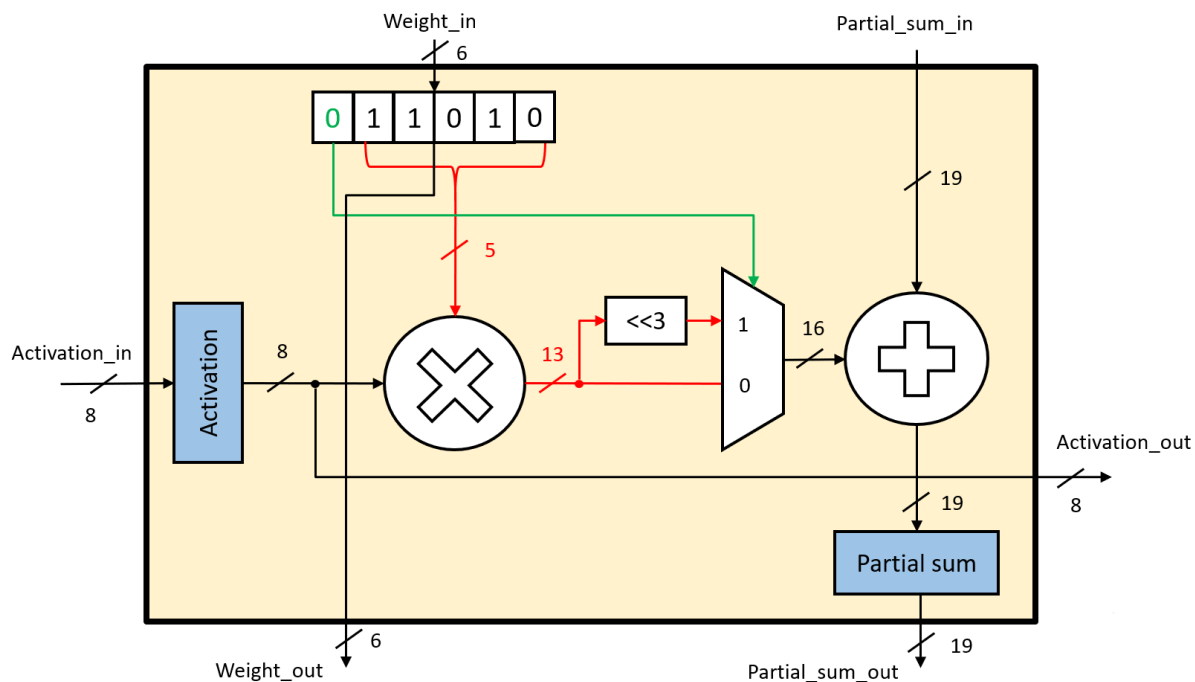


圖 4.5 第一型態的降低位元精確度之處理單元之示意圖

接下來我們將第一型態的降低位元精確度之處理單元以脈動陣列的形式連接起來，如圖 4.6 所示，整體的架構基本上跟傳統的張量處理器一樣，雖然都是以處理單元進行運算並傳遞到下方的累加器做部分和的累加，但是每個處理單元內部的面積卻降低了不少，隨著脈動陣列的大小增加，硬體成本的差距跟傳統的張量處理器相比也會跟著變大，雖然將權重值存進權重記憶體之前都必須經過權重預處理單元來縮減權重值的精確度，但即使我們使用了權重預處理單元，其硬體成本跟整個脈動陣列相比也只是微乎其微。

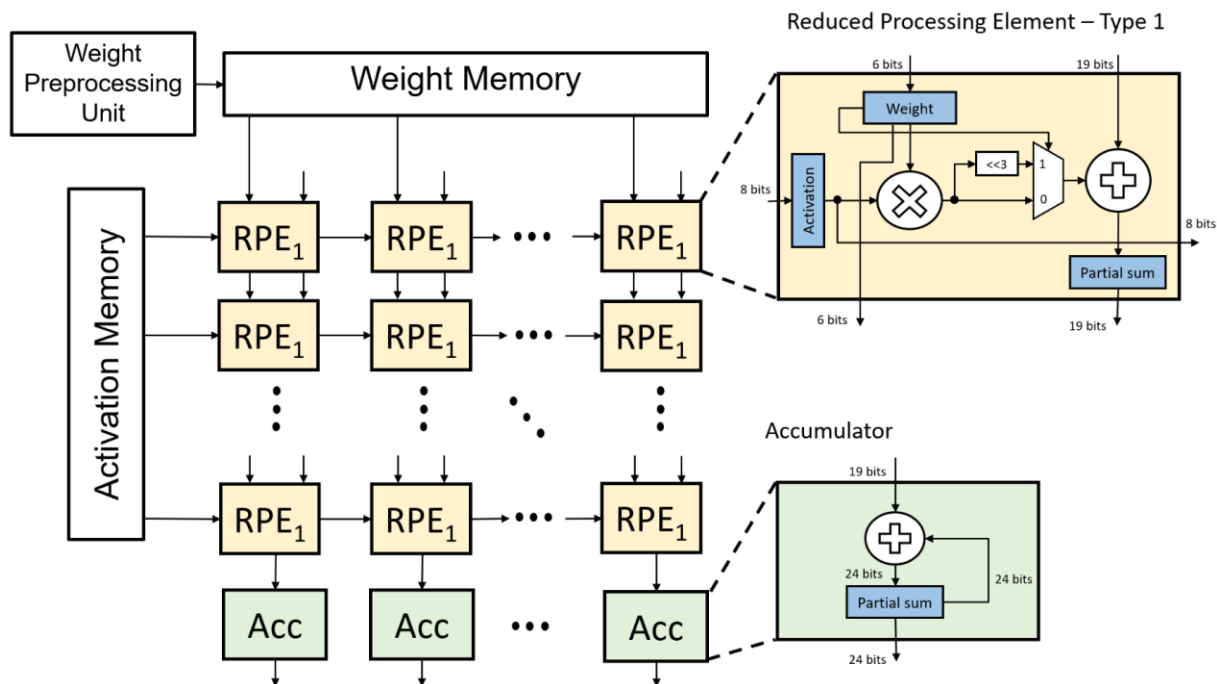


圖 4.6 第一型態的降低位元精確度之張量處理器之示意圖

雖然第一型態的張量處理器可以藉由較低的硬體成本來完成深度學習的任務，但是在執行大型的深度學習模型時，準確度的結果卻不盡理想，雖然根據權重值的觀察發現，有 99 % 的權重值可以藉由重要的連續位元來讓權重值的精確度在無準確度損失的情況下縮減，但是在權重數量龐大的情況下將會使準確度出現誤差，舉例來說，在一千萬個權重值之中即使只有 1 % 的權重值有準確度的損失，那就相當於十萬個權重值會造成準確度的結果出現變化，因此造成準確度大幅降低，然而 8 位元精確度的權重值跟 16 位元精確度相比，無條件捨棄了更精確的數值。

為了提升整體的準確度，我們必須在 16 位元精確度的情況下，使用近似值來讓 8 位元精確度的權重可以更接近 16 位元精確度，於是我們將權重值以 16 位元的精確度去進行分析，觀察權重值最低的 9 個位元的分佈狀況，如圖 4.7 所示，我們使用累積分佈函數 (Cumulative Distribution Function, CDF) 觀察這 9 個位元的累積分佈情形，可以看到將近一半的權重值都小於 9 位元的 100000000，而另一半的權重值則大於 9 位元的 100000000，這也就代表著在機率平均分佈的情況

下，所有權重的期望值大約會等於 9 位元的 100000000，因此相當於在 16 位元精確度的權重值裡，每個權重值中最低的 8 個位元皆為 0，而第 9 個位元皆為 1，所以捨棄最低的 8 個位元後，就形成了相當於 16 位元精確度的 8 位元權重值，且這些權重值的最低位元 (Least Significant Bit, LSB) 皆為 1，我們依照動態範圍的無偏乘法器 [2] 的方法去計算被捨棄掉的位元期望值，發現在 8 位元精確度的權重值中，將最低的位元設為 1 剛好可以當作近似值，來取代 16 位元的精確度，並提升整體的準確度，後續將介紹第二型態的降低位元精確度之張量處理器架構。

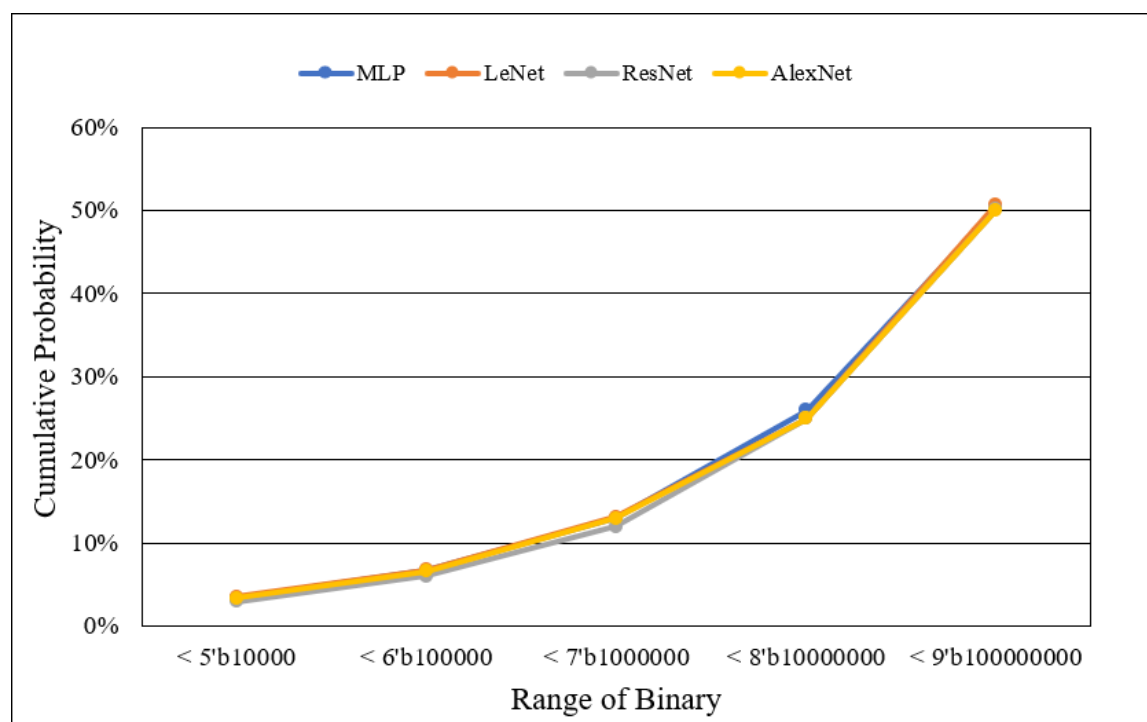


圖 4.7 精確度 16 位元的權重中最低的 9 位元資料累積分佈

4.4.2 第二型態的降低位元精確度之張量處理器架構

由權重值累積分佈的結果發現，我們只要將權重值最低的位元設為 1，剛好就能當作期望值來取代那些被捨棄掉的權重位元，並提高原本失去的準確度，於是我們將此概念套用在本篇論文所提出的張量處理器中，如圖 4.8 所示，為第二型態的降低位元精確度之處理單元架構，由於準確度可以透過期望值進行彌補，我們將激活值的字長也縮短成 5 位元，在輕微失去準確度的情況下進一步減少面

積成本，但是跟本篇論文提出的第一型態的處理單元相比，可以同時達到較高的準確度以及較低的硬體成本。

從權重值累積分佈的結果發現，既然每個權重值最低的位元都必須設為 1 來補償失去的權重位元的準確度，那麼在硬體架構方面，我們只要將權重值暫存器與激活值暫存器的最低位元固定為 1，就可以使兩者輸入各自減少 1 個位元的傳輸，如圖上所示，激活值與權重值的輸入精確度分別為 4 位元與 5 位元，由於先前提到重要的連續位元使得權重值產生兩種情形的縮減，因此輸入權重值跟激活值相比多了 1 個位移位元來判斷是否需要進行移位，而實際送入乘法器計算的權重值與激活值皆為 5 位元的精確度，由於第二型態的降低位元精確度之處理單元將所有的激活值中最低的 3 個位元做捨棄，因此每個乘法計算後的結果都必須左移 3 次來讓計算結果近似於原本的值，而權重值的位移位元同樣需要判斷是否進行左移 3 次，因此我們將權重值與激活值的移位次數合併在一起考慮後得出判斷的結果，當位移位元為 0 時，乘法計算的結果需要左移 3 次；當位移位元為 1 時，乘法計算的結果需要左移 6 次。

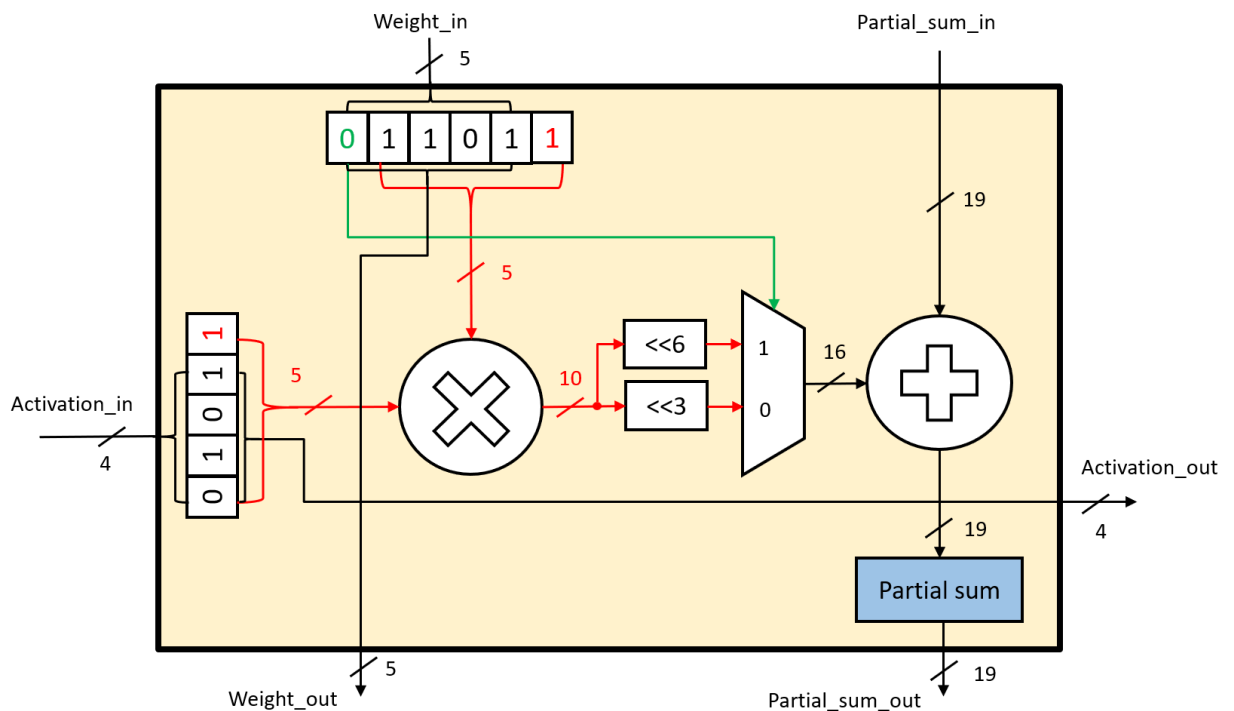


圖 4.8 第二型態的降低位元精確度之處理單元之示意圖

我們將第二型態的降低位元精確度之處理單元以脈動陣列的形式連接起來後，組成了第二型態的降低位元精確度之張量處理器架構，與第一型態相比，每個處理單元的面積又縮小了一些，由於期望值對於準確度的補償，使得激活值從原本 8 位元的精確度縮減成 5 位元的精確度時，不會讓準確度大幅的降低，因此整體的硬體成本比第一型態的降低位元精確度之張量處理器優化了許多。

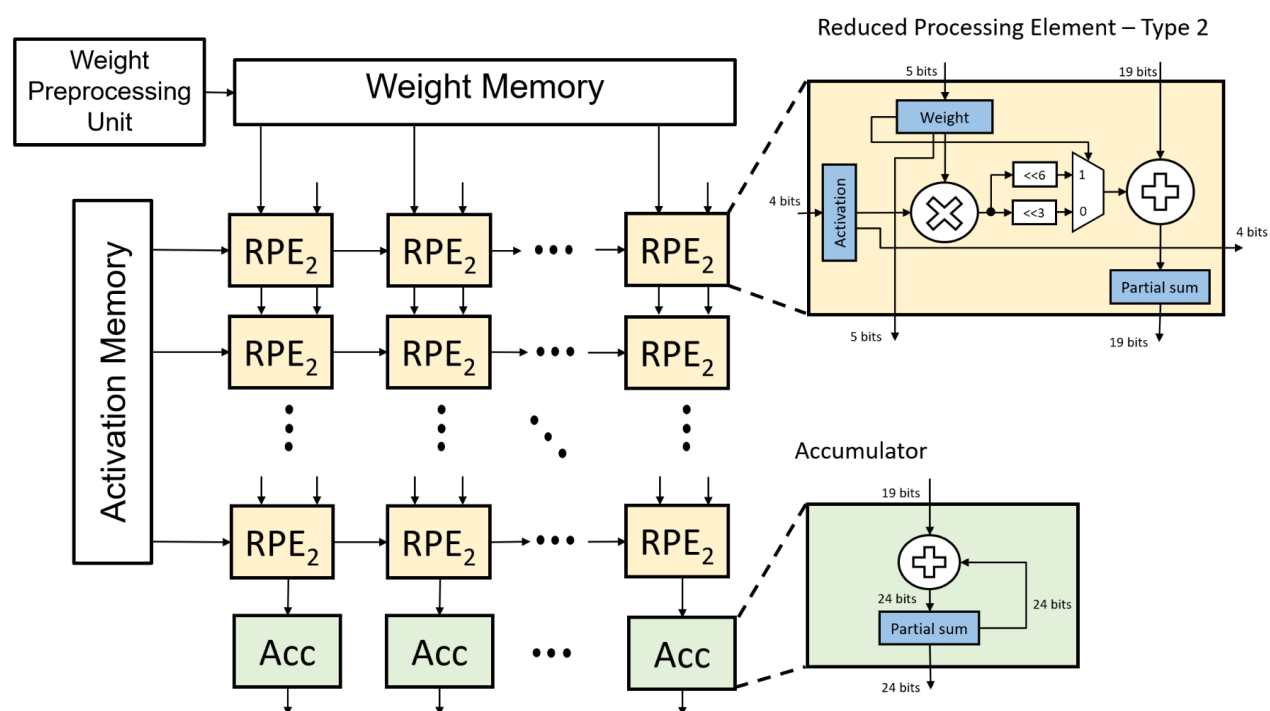


圖 4.9 第二型態的降低位元精確度之張量處理器之示意圖

如圖 4.10 所示，為降低位元精確度之張量處理器架構，當權重值從外部輸入進來時，會經過權重預處理單元來縮短權重值的精確度，並寫入權重記憶體之中。架構中的統一緩衝器可以儲存外部的輸入資料或是脈動陣列計算神經網路的每一層資料，並藉由一個多工器來選擇路徑，統一緩衝器的資料可以存進激活記憶體，當作激活值的資料來源，當神經網路運算開始執行時，權重記憶體與激活記憶體會將先前儲存的資料送入脈動陣列中，而脈動陣列則為本篇論文使用的兩種型態的處理單元來進行近似運算，第 i 型態的降低位元精確度之處理單元以 $RPE_i, i \in 1, 2$ 表示，將脈動陣列計算完的資料由下方的累加器進行累加動作，直到所有的計算皆累加完畢後，累加器的輸出結果會經過激活函數進行非線性的轉

換，使神經網路模型可以提高靈活度，以便處理更複雜的數據，最後將轉換後的資料存進統一緩衝器，並準備進行下一層的運算，依此類推，重複上述的動作，依序完成所有的深度神經網路計算任務。

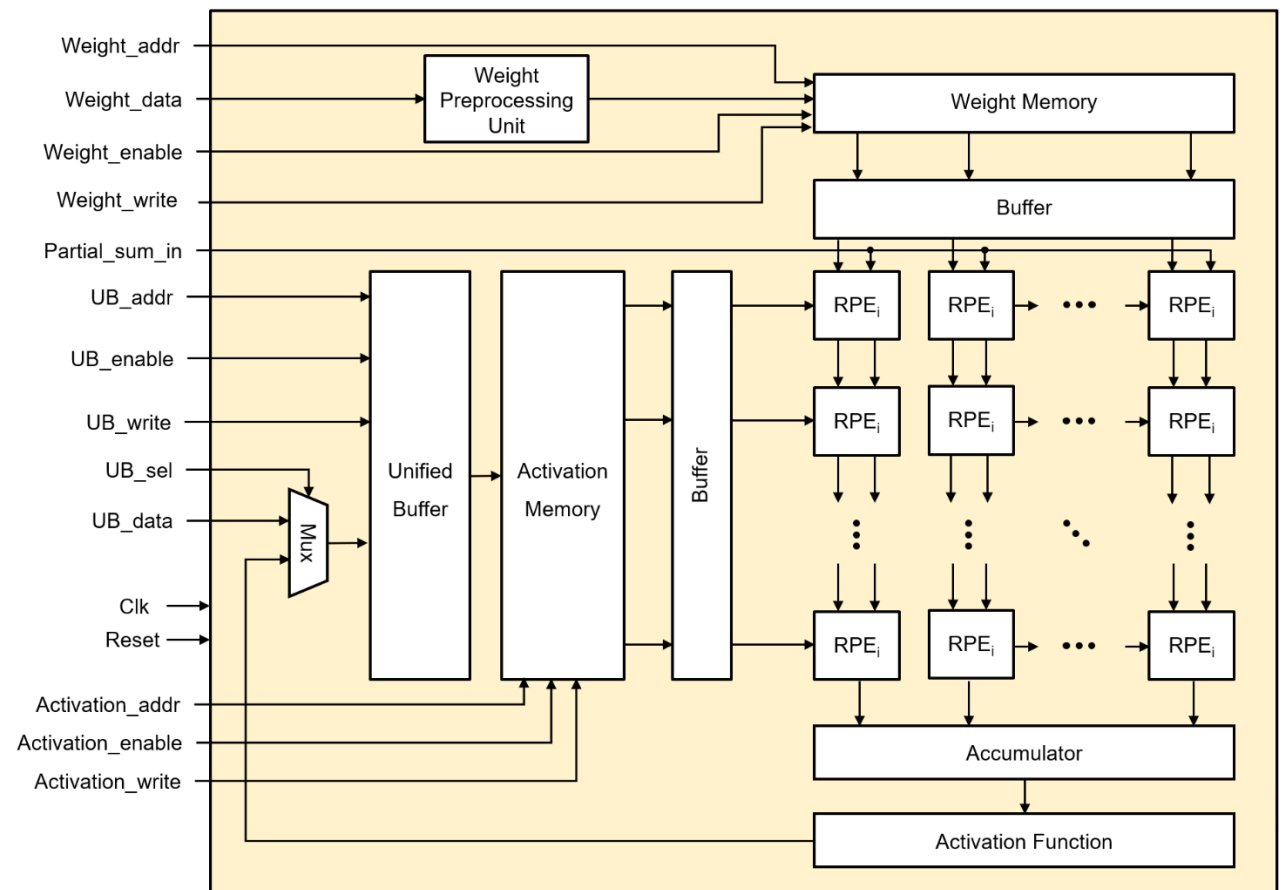


圖 4.10 降低位元精確度之張量處理器架構

第五章 實驗結果

5.1 深度學習模型設定

本篇論文使用 Pytorch [32] 深度學習框架為平台來模擬實驗結果，使用深度神經網路模型包含：

- (1) MLP [13]: 由三層全連結神經網路所組成的深度神經網路架構，輸入層由 784 個神經元所組成，輸出層則由 10 個神經元組成，使用資料集為 MNIST [33] 手寫辨識資料集，深度神經網路訓練完成後的準確度為 95.4 %。
- (2) LeNet [14]: 此深度學習由五層神經網路所組成，是早期卷積神經網路中典型的 CNN 網路架構，前三層為卷積神經網路，後兩層為全連結神經網路，使用資料集為 MNIST [33] 手寫辨識資料集，深度神經網路訓練完成後的準確度為 98.8 %。
- (3) ResNet [15]: 此深度學習由十八層神經網路架構所組成，含有十六層的卷積神經網路與一層的全連結神經網路，還有一層預處理的卷積層，但是不包含在計算內，使用資料集為 MNIST [33] 手寫辨識資料集，深度神經網路訓練完成後的準確度為 97.9 %。
- (4) AlexNet [16]: 此深度學習由八層神經網路架構所組成，有五層的卷積神經網路與三層的全連結神經網路，為深度學習重要的里程碑，幫助圖像辨識取得了重大的突破，使用資料集為 MNIST [33] 手寫辨識資料集，深度神經網路訓練完成後的準確度為 97 %。

如表 5.1 所示，為各模型之參數整理，其中包含層數 (Layer)、資料集 (Dataset)、輸入尺寸 (Input Dimensions)、輸出分類 (Output Classes) 和訓練的準確度 (Trained Accuracy)，其中層數包含卷積神經網路層數 (CONV) 與全連結神經網路層數 (FC)。

表 5.1 神經網路模型之參數

Module Name	MLP	LeNet	ResNet	AlexNet
Layers (CONV/FC)	3 (0/3)	5 (2/3)	17 (16/1)	8 (5/3)
Dataset	MNIST	MNIST	MNIST	MNIST
Input Dimensions	28×28	28×28	28×28	28×28
Output Class	10	10	10	10
Trained Accuracy	95.4%	98.8%	97.9%	97.0%

5.2 準確度分析

本章節在四個深度神經網路模型之下，各別分析兩種型態的降低位元精確度之張量處理器之準確度，我們將權重值設定在不同的精確度之下，並且互相比較各個神經網路模型的準確度，最後將觀察到的結果以折線圖的方式呈現出來，並用表格紀錄所有數值。

圖 5.1 為第一型態的降低位元精確度之張量處理器準確度分析，從圖上可以看到在權重值的精確度為 5 位元時，MLP 與 LeNet 的神經網路模型仍然可以得到 90 % 以上的準確度，由於這兩個模型較小且權重的數量少，即使每個權重值的精確度都出現了一些誤差，但是這些少量的權重誤差累積起來並不足以讓神經網路模型的判斷出現錯誤，因此損失的準確度跟其他模型相比之下較少；而 ResNet 與 AlexNet 的神經網路模型在第一型態的降低位元精確度之張量處理器中，不管使用哪一種精確度的權重值，其精確度都在 10 % 左右，因為這兩個模型較大且權重的數量多，即使每個權重值的精確度誤差只有一些，但是眾多權重的誤差累積起來將導致準確度大幅的下降，因此損失的準確度跟其他模型相比之下較多。表 5.2 統整了第一型態的降低位元精確度之張量處理器在不同權重精確度之下的準確度。

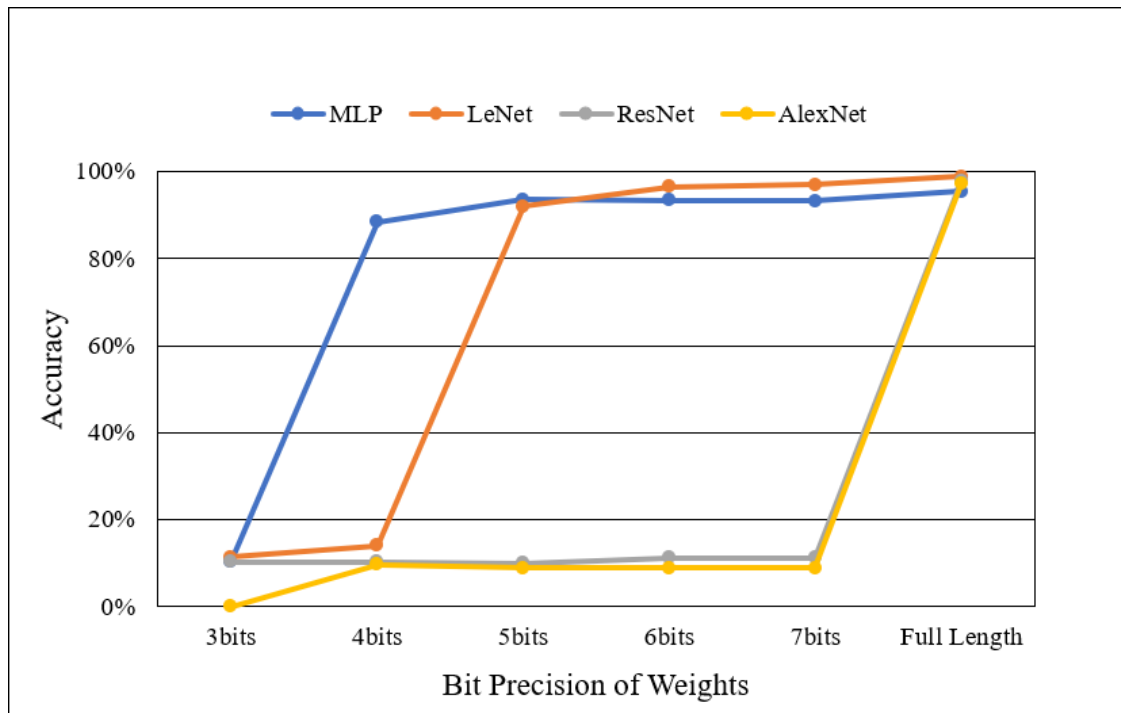


圖 5.1 第一型態的降低位元精確度之張量處理器準確度分析

表 5.2 第一型態的降低位元精確度之張量處理器之準確度參數

Module Name Bit Precision	MLP	LeNet	ResNet	AlexNet
3 bits	10.5%	11.4%	10.3%	0.0%
4 bits	88.5%	14.0%	10.3%	9.7%
5 bits	93.6%	92.0%	10.0%	8.9%
6 bits	93.5%	96.6%	11.3%	8.9%
7 bits	93.3%	97.0%	11.3%	8.9%
Full Length	95.4%	98.9%	97.8%	97.1%

圖 5.2 為第二型態的降低位元精確度之張量處理器準確度分析，從圖上可以看到在權重值的精確度為 4 位元時，每個深度神經網路模型的準確度分析仍然有 76 % 以上，因為第二型態的降低位元精確度之張量處理器彌補了第一型態損失的準確度，藉由計算期望值來取代那些被精確度捨棄的位元，使得字長縮減後的權重值仍然可以近似於原本較高的精確度，因此在低精確度的情況下，神經網路的推論結果仍然可以維持一定的準確度。表 5.3 統整了第二型態的降低位元精確度之張量處理器在不同精確度之下的準確度，可以看到在不同位元的精確度之下，

其準確度與表 5.2 的結果相比提升了許多，代表第二型態的降低位元精確度之張量處理器解決了第一型態的缺點。

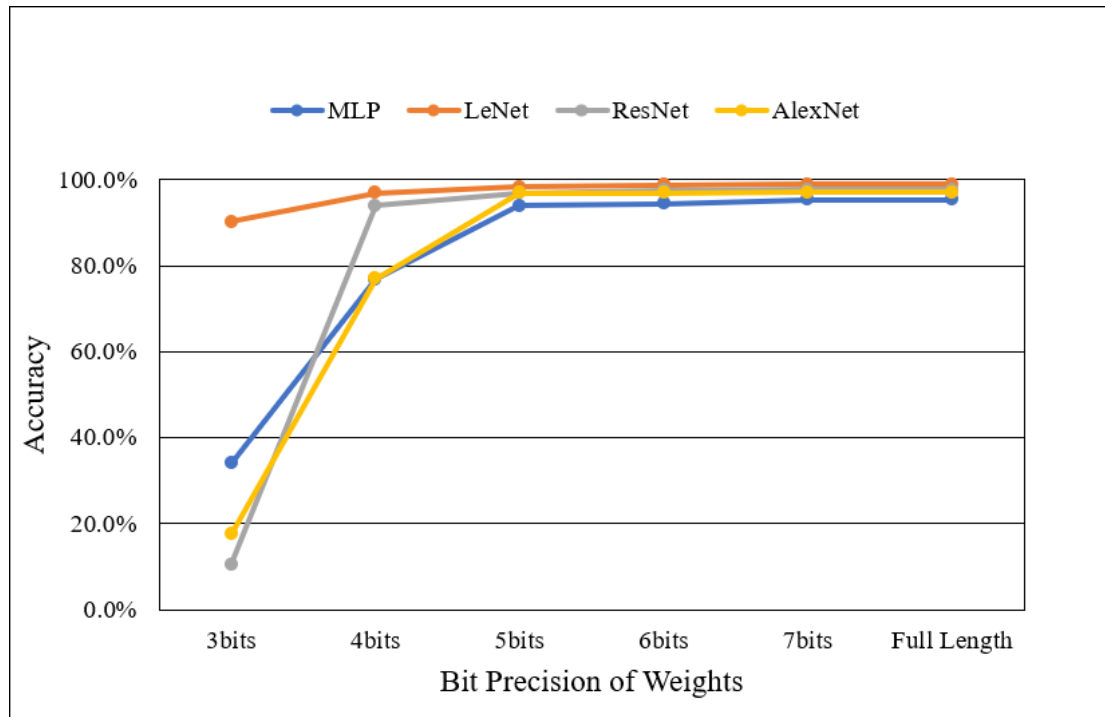


圖 5.2 第二型態的降低位元精確度之張量處理器準確度分析

表 5.3 第二型態的降低位元精確度之張量處理器之準確度參數

Module Name Bit Precision	MLP	LeNet	ResNet	AlexNet
3 bits	34.0%	90.3%	10.6%	17.5%
4 bits	76.9%	97.0%	94.0%	77.0%
5 bits	94.0%	98.3%	97.0%	97.0%
6 bits	94.5%	98.8%	97.6%	97.0%
7 bits	95.4%	98.8%	97.7%	97.1%
Full Length	95.4%	98.9%	97.8%	97.1%

最後我們將本篇論文的第一型態與第二型態的降低位元精確度之處理單元與其他相關論文研究 [1, 2, 3] 的處理單元做比較，並且輸入資料都以定點數的格式做準確度分析，如表 5.4 所示，我們將傳統的處理單元在不同神經網路模型之下皆以全字長 (Full Length) 的精確度來模擬，確保它的準確度維持在一定的

水平，其餘的處理單元皆使用縮減過後的精確度來觀察準確度的影響程度，可以看到在 MLP、LeNet 的深度神經網路模型之下，張量處理器使用任何一種處理單元都可以達到 88 % 以上的準確度，而在 AlexNet 這種大型的深度神經網路模型之下，大部分的處理單元都出現準確度大幅降低的情況，甚至在 ResNet 的模型中，第一型態的降低位元精確度之處理單元的準確度只有 9.8 %，而本篇論文提出的第二型態的降低位元精確度之處理單元與其他處理單元相比都有較高的準確度，因為第二型態的降低位元精確度之處理單元不僅利用了重要的連續位元來讓權重值在無準確度損失的情況下進行縮減，同時利用那些被捨棄的權重位元的期望值，來補償縮減後的權重值所失去的精確度，因此讓本篇論文的處理單元同時具有準確度高且面積成本較少的優點。

表 5.4 與其他相關論文之準確度比較

Types of PE	Precision	Module Name			
		MLP	LeNet	ResNet	AlexNet
PE [1]	Full Length	95.4%	98.9%	97.8%	97.1%
DRUM PE [2]	5 bits $\times$ 5 bits	90.0%	88.4%	91.5%	58.5%
APE [3]	5 bits $\times$ 5 bits	90.0%	88.4%	91.5%	58.5%
RPE ₁	8 bits $\times$ 5 bits	93.6%	95.5%	9.8%	8.9%
RPE ₂	5 bits $\times$ 5 bits	94.0%	98.1%	97.8%	96.7%

5.3 硬體成本分析

此小節主要分析本篇論文提出的第一型態與第二型態的降低位元精確度之處理單元與其他相關論文的面積成本與功率消耗，我們以傳統的處理單元為基準去比較面積之間的差異，第一個相關的論文研究 [2] 藉由縮減輸入精確度的方式來縮小乘法器的大小，並計算被捨棄的所有資料中的期望值，來避免乘法結果出現較大的誤差，而另一個相關的論文研究 [3] 藉由預近似單元來縮減處理單元輸入的資料精確度，並且記錄位移次數與符號位元給後續的處理單元做計算，

每個處理單元只需要將字長縮減後的資料相乘並進行移位即可，因此處理單元內部的元件數量減少，使得張量處理器的面積可以進一步的縮小，而本篇論文藉由權重值重要的連續位元來縮減權重的字長，並且將每個權重值存入記憶體前就進行精確度的縮減，使得後續的硬體不需要再進行資料處理的動作，本篇論文的硬體面積跟其他論文 [2, 3] 相差不遠，卻可以利用更少的功率消耗來完成處理單元內的計算。

如表 5.5 所示，我們統整了本篇論文與其他相關論文的比較結果，在 MLP 深度學習模型之下的準確度分析，本篇論文的第一型態與第二型態的降低位元精確度之處理單元跟傳統的處理單元相比只降低了 2 % 以內，由於處理單元內部乘法器的縮小，兩種型態的處理單元分別下降了 34 % 與 43 % 的面積，而更小的乘法器可以加快硬體的計算速度，其操作頻率的效能也提升了 1.15 倍至 1.19 倍，且功率消耗也能降低 18 % 至 27 %。本篇論文與其他相關論文 [2, 3] 相比，在面積大小相差不多的情況下，其操作頻率也能提升將近 1.15 倍以上的效能，且降低 10 % 至 34 % 的功率消耗。

表 5.5 與其他相關論文之綜合結果比較

Types of PE	PE [1]	DRUM PE [2]	APE [3]	RPE ₁	RPE ₂
Accuracy	95.4%	90%	90.0%	93.6%	94.0%
Frequency (MHz)	200	170	200	230	238
Area	1x	0.8x	0.55x	0.66x	0.57x
Power (mW)	0.11	0.12	0.1	0.09	0.08
GOPS	25.6	21.8	25.6	29.4	30.4

5.4 張量處理器之數位電路模擬

在這一小節，將針對本篇論文的第一型態與第二型態的降低位元精確度之張量處理器進行電路模擬，以便觀察輸出的計算結果是否正確，本架構是由第一型態與第二型態的降低位元精確度之處理單元所組成的 8×8 脈動陣列進行模擬，

並觀察輸入的激活值與權重值相乘與移位的情形。

圖 5.3 為第一型態的降低位元精確度之處理單元之電路模擬，我們分別將輸入的激活值與權重值各送入正數值與負數值，因此有 4 種情形，由於權重值是藉由重要的連續位元來縮減精確度後的資料，因此要觀察權重值的最高位元 (Most Significant Bit, MSB)，也就是觀察位移位元來判斷是否需要進行移位，每當進行乘法運算且權重值的最高位元為 1 時，就將計算結果左移 3 次，相當於乘法結果再乘以 8；而權重值的最高位元為 0 時，計算結果將不進行移位，最後再加上輸入端送入的部分和，此範例的輸入為 3，因此計算後的結果需要加 3。

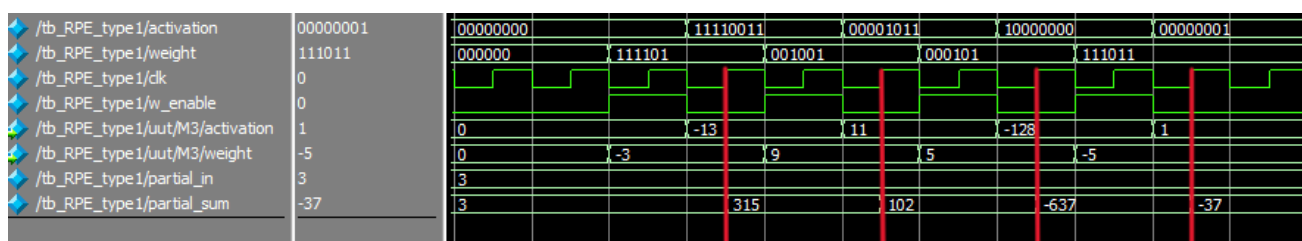


圖 5.3 第一型態的降低位元精確度之處理單元之電路模擬

圖 5.4 為第一型態的降低位元精確度之張量處理器之電路模擬範例一，由於電路處理單元數量多，因此我們只觀察 8×8 脈動陣列的第一行處理單元的輸出結果，範例將每個處理單元的激活值與權重值分別送入數值 8 與 1，由於權重值的最高位元為 0，因此計算後不進行移位，並往下繼續累加，每個處理單元的乘法結果為 8，因此 8 個處理單元的累加結果為 64。

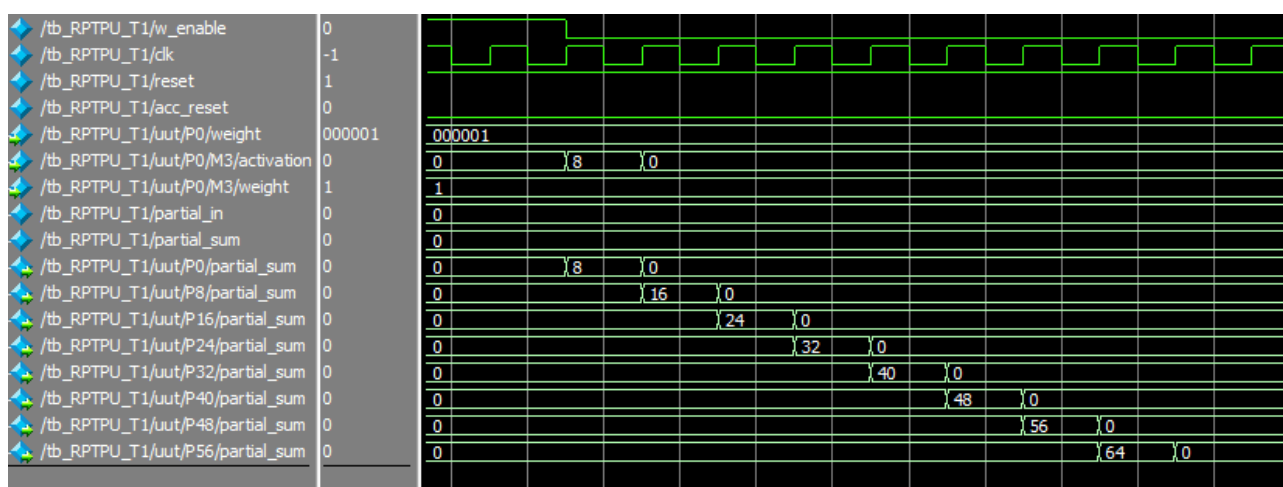


圖 5.4 第一型態的降低位元精確度之張量處理器之電路模擬範例一

圖 5.5 為第一型態的降低位元精確度之張量處理器之電路模擬範例二，範例將每個處理單元的激活值與權重值分別送入數值 8 與 -1，由於權重值的最高位元為 1，因此乘法計算後皆須進行左移 3 次，並往下繼續累加，每個處理單元的乘法結果為 -64，因此 8 個處理單元的累加結果為 -512。

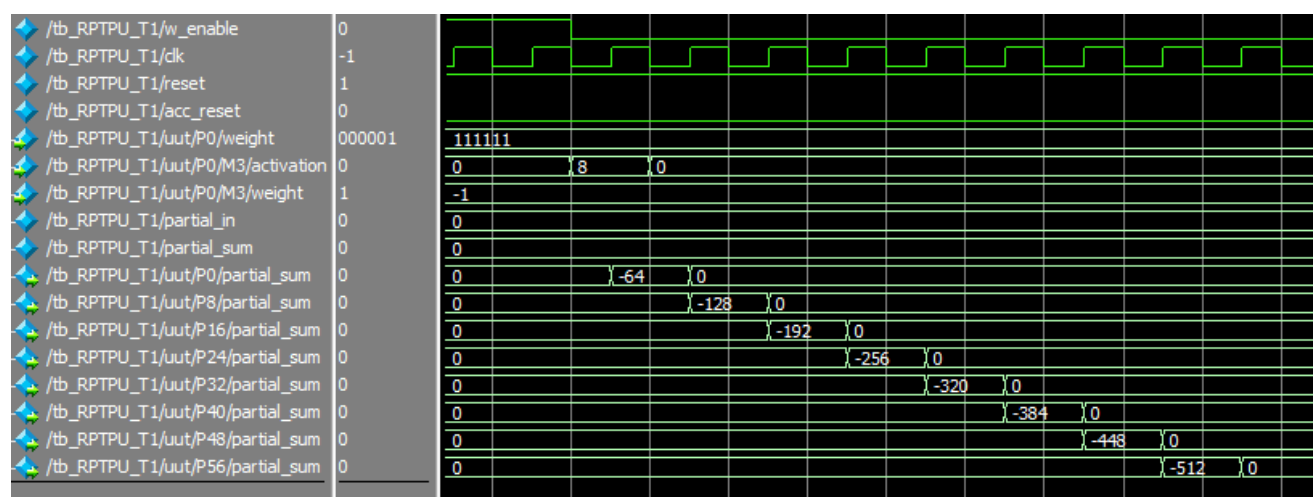


圖 5.5 第一型態的降低位元精確度之張量處理器之電路模擬範例二

圖 5.6 為第二型態的降低位元精確度之處理單元之電路模擬，由於第二型態的處理單元中，激活值的精確度皆少了 3 位元，跟第一型態相比需要將計算結果額外進行 3 次左移的動作，因此每當進行乘法運算且權重值的最高位元為 1 時，計算結果總共需要左移 6 次，相當於乘法結果再乘以 64；而權重值的最高位元為 0 時，則將計算結果左移 3 次，最後再加上輸入端送入的部分和，此範例的輸入為 3，因此計算後的結果需要加 3。

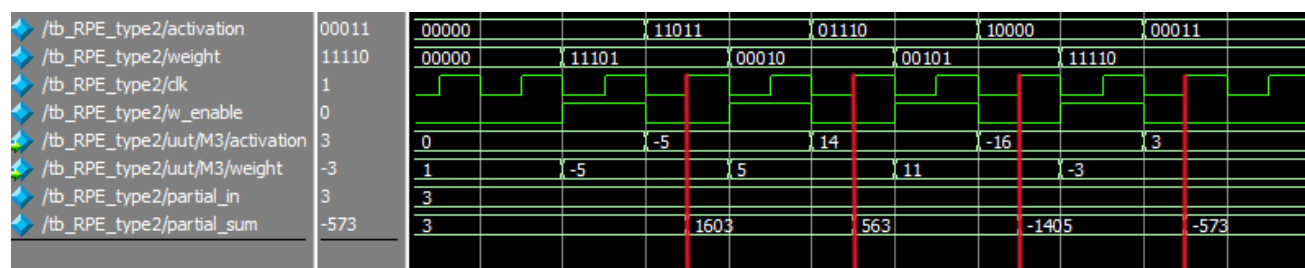


圖 5.6 第二型態的降低位元精確度之處理單元之電路模擬

圖 5.7 為第二型態的降低位元精確度之張量處理器之電路模擬範例一，與第一型態相同，我們只觀察 8×8 脈動陣列的第一行處理單元的輸出結果，範例將每個處理單元的激活值與權重值分別送入數值 8 與 3，由於權重值的最高位元為 0，因此計算後的結果皆左移 3 次，並往下繼續累加，每個處理單元的乘法結果為 192，因此 8 個處理單元的累加結果為 1536。

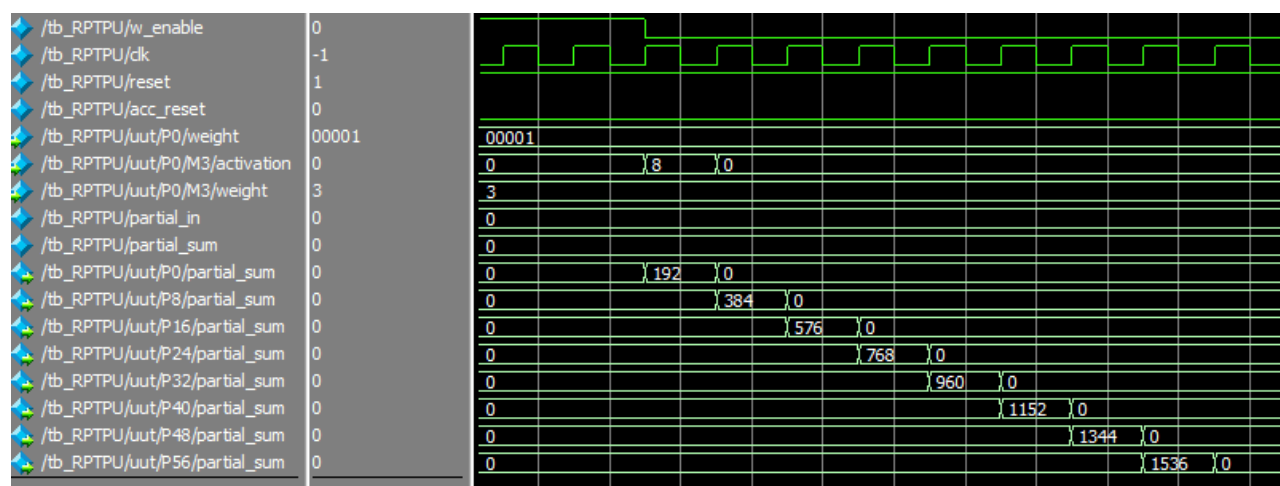


圖 5.7 第二型態的降低位元精確度之張量處理器之電路模擬範例一

圖 5.8 為第二型態的降低位元精確度之張量處理器之電路模擬範例二，範例將每個處理單元的激活值與權重值分別送入數值 8 與 -1 ，由於權重值的最高位元為 1，因此乘法計算後皆須進行左移 6 次，並往下繼續累加，每個處理單元的乘法結果為 -512 ，因此 8 個處理單元的累加結果為 -4096 。

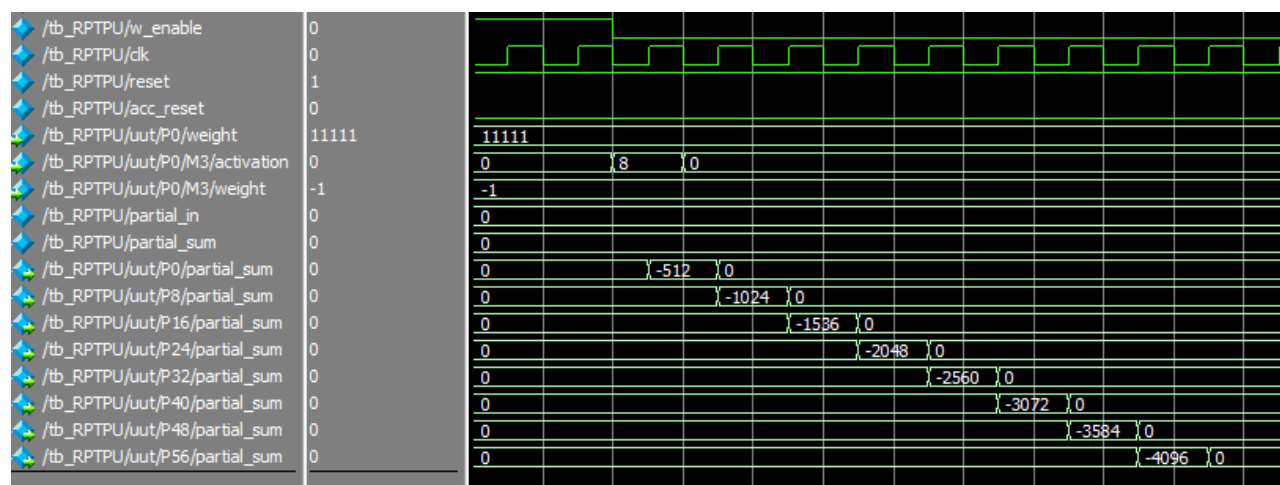


圖 5.8 第二型態的降低位元精確度之張量處理器之電路模擬範例二

5.5 超大型積體電路實現

本論文使用 Cell-Based Design 的 ASIC 設計流程來實現張量處理器架構，使用製程為 TSMC Artisan 90 nm 1P9M 與標準元件資料庫 (Standard Cell Library) 進行實現，如表 5.6 所示，為張量處理器的超大型積體電路實現結果，電路操作頻率可以達到 238 MHz，其中激活記憶體與權重記憶體分別為 32 KB 與 40 KB，加上一個 32 KB 的統一緩衝器，總共佔整體電路的 90.2%，在邏輯電路的部分，張量處理器的脈動陣列與累加器分別佔整體電路的 7.4% 與 0.48%，而其餘的權重預處理單元與激活函數轉換元件等周邊電路約佔整體電路的 2%。

表 5.6 張量處理器的超大型積體電路實現結果

Design Specifications	
Process Technology	TSMC 90 nm 1P9M
Frequency	238 MHz
Activation memory size	32 KB
Weight memory size	40 KB
Unified buffer size	32 KB
Total gate count	487678 (100%)
Memory gate count	439893 (90.2%)
Systolic array gate count	35817 (7.4%)
Accumulator gate count	2354 (0.48%)
Function gate count	8153 (1.67%)
WPU gate count	105 (0.02%)
Miscellaneous	1356 (0.23%)
Core Area	2,036,329 μm^2
Chip Area	3,873,024 μm^2

圖 5.3 為本架構之電路佈局圖，供應電壓為 1.8 V，核心面積的大小為 $2,036,329 \mu\text{m}^2$ ，而晶片的大小為 $3,873,024 \mu\text{m}^2$ ，圖中右上方為權重記憶體與激活記憶體，左方為統一緩衝器，右下方為張量處理器的 8×8 脈動陣列與累加器電路，而其餘的則是權重預處理單元與激活函數轉換元件等周邊電路。

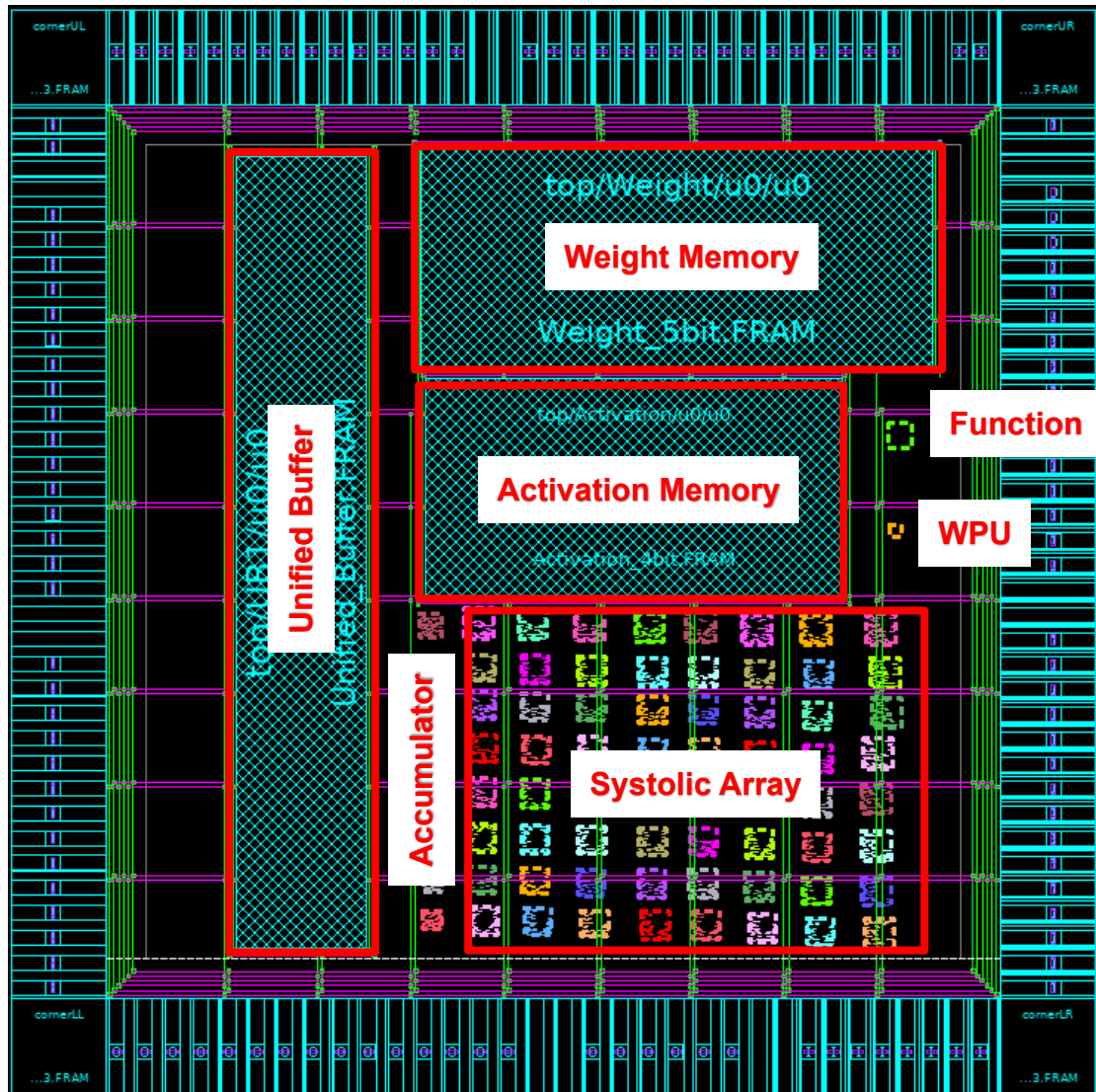


圖 5.9 第二型態的降低位元精確度之張量處理器之佈局圖

第六章 結論與未來展望

6.1 結論

本篇論文提出的降低位元精確度之張量處理器，針對傳統的張量處理器進行低成本的改良，由於大部分的權重值可以在不造成準確度下降的情況下縮減字長，使得張量處理器中的處理單元內部可以使用更小的乘法器，因此第一型態的降低位元精確度之張量處理器與第二型態的降低位元精確度之張量處理器的硬體成本跟傳統的張量處理器相比分別下降了 34 % 與 43 %，而更小的乘法器可以加快硬體的計算速度，其操作頻率的效能也提升了 1.15 倍至 1.19 倍，且功率消耗也能降低 18 % 至 27 %。本篇論文與其他相關論文 [2, 3] 相比之下，其操作頻率相比也能提升將近 1.15 倍以上的效能，且降低 10 % 至 34 % 的功率消耗，在 MLP 模型的準確度相比之下，本篇論文在相差不遠的硬體成本中準確度也高了 3 % 以上。

6.2 未來展望

本篇論文提出的低成本技術主要是針對張量處理單元內部的電路進行精確度的縮減，以達到硬體成本與功耗的降低，未來可以考慮結合其他低功耗的架構，進一步節省整體電路的能量消耗，或是在處理單元內部使用更高準確度的近似運算，來提高深度神經網路的準確度。

參考文獻

- [1] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, N. Boden, A. Borchers, and R. Boyle, “In-Datacenter Performance Analysis of a Tensor Processing Unit,” in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, pp. 3–4, June 2017.
- [2] S. Hashemi, R. I. Bahar, and S. Reda, “DRUM: A Dynamic Range Unbiased Multiplier for Approximate Applications,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design*, pp. 419–420, Nov. 2015.
- [3] Mohammed E. Elbity, Peyton S. Chandarana, Brendan Reidy, Jason K. Eshraghian, and Ramtin Zand, “APTPU: Approximate Computing Based Tensor Processing Unit,” *IEEE Trans. Circuits Syst. I*, vol. 69, no. 12, pp. 5135–5144, Dec. 2022.
- [4] W. A. Wulf and S. A. McKee, “Hitting the Memory Wall: Implications of the Obvious,” *ACM SIGARCH Comput. Archit. News*, vol. 23, no. 1, pp. 20–24, Mar. 1995.
- [5] M. Valle, “Analog VLSI Implementation of Artificial Neural Networks with Supervised On-Chip Learning,” *Analog Integr. Circuits and Signal Process.*, vol. 33, pp. 263–287, Dec. 2002.
- [6] M. Bouvier, A. Valentian, T. Mesquida, F. Rummens, M. Reyboz, E. Vianello, and E. Beigne, “Spiking Neural Networks Hardware Implementations and Challenges: A Survey,” *ACM J. on Emerg. Technol. in Comput. Syst.*, vol. 15, no. 2, pp. 1–35, Apr. 2019.
- [7] Y. Chen, Y. Xie, L. Song, F. Chen, and T. Tang, “A Survey of Accelerator Architectures for Deep Neural Networks,” *Engineering*, vol. 6, no. 3, pp. 264–274, Mar. 2020.

- [8] S. Mittal, “A Survey on Optimized Implementation of Deep Learning Models on the NVIDIA Jetson Platform,” *Journal of Syst. Archit.*, vol. 97, pp. 428–442, Aug. 2019.
- [9] L. Deng, J. Li, J. T. Huang, K. Yao, D. Yu, F. Seide, M. Seltzer, G. Zweig, X. He, J. Williams, Y. Gong, and A. Acero, “Recent Advances in Deep Learning for Speech Research at Microsoft,” in *Proc. IEEE Int. Conf. on Acoustics, Speech and Signal Processing*, pp.8604–8608, May 2013.
- [10] P. J. Bannon and K. A. Hurd, “Accelerated Mathematical Engine,” U.S. Patent 0026078 A1, Jan.2019.
- [11] P. Gysel, M. Motamedi, and S. Ghiasi, “Hardware-oriented Approximation of Convolutional Neural Networks,” *arXiv preprint arXiv:1604.03168*, pp. 1–10, Oct. 2016.
- [12] T. F. Hsieh, J. F. Li, J. S. Lai, C. Y. Lo, D. M. Kwai, and Y. F. Chou, “Refresh Power Reduction of DRAMs in DNN Systems Using Hybrid Voting and ECC Method,” in *Proc. IEEE International Test Conference in Asia*, pp. 41–46, Sep. 2020.
- [13] H. Ramchoun, M. A. Idrissi, Y. Ghanou, and M. Ettaouil, “Multilayer Perceptron: Architecture Optimization and Training with Mixed Activation Functions,” in *Proc. of the 2nd Int. Conf. on Big Data, Cloud and Applications*, pp. 1–6, Mar.2017.
- [14] M. Kayed, A. Anter, and H. Mohamed, “Classification of Garments from Fashion MNIST Dataset Using CNN LeNet-5 Architecture,” *International Conference on Innovative Trends in Communication and Computer Engineering*, pp. 238–243, Feb. 2020.
- [15] M. F. Haque, H. Y. Lim, and D. S. Kang, “Object Detection Based on VGG with ResNet Network,” in *Proc. International Conference on Electronics, Information, and Communication*, pp. 1–3, Jan 2019.

- [16] T. Shanthi and R. S. Sabeenian, "Modified Alexnet Architecture for Classification of Diabetic Retinopathy Images," *Computers & Electrical Engineering*, pp. 56–64, Jun. 2019.
- [17] N. Brunel, V. Hakim, and M. J. E. Richardson, "Single Neuron Dynamics and Computation," *Current Opinion in Neurobiology*, vol. 25, pp. 149–155, Apr. 2014.
- [18] X. Yin, J. Goudriaan, E. A. Lantinga, J. Vos, and H. J. Spiertz, "A Flexible Sigmoid Function of Determinate Growth," *Ann. Botany*, vol. 91, no. 3, pp. 361–371, Feb. 2003.
- [19] M. M. Lau and K. H. Lim, "Review of Adaptive Activation Function in Deep Neural Network," in *Proc. IEEE-EMBS Conf. Biomed. Eng. Sci.*, pp. 686–690, Dec. 2018.
- [20] X. Glorot, A. Bordes, and Y. Bengio, "Deep Sparse Rectifier Neural Networks," in *Proc. 14th Int. Conf. Artif. Intell. Statist.*, vol. 15, pp. 315–323, Apr. 2011.
- [21] A. Schwing and R. Urtasun, "Fully Connected Deep Structured Networks," *arXiv:1503.02351*, pp. 1–10, Mar. 2015.
- [22] C. Nebauer, "Evaluation of Convolutional Neural Networks for Visual Recognition," *IEEE Transactions on Neural Networks*, vol. 9, no. 4, pp. 685–696, Jan. 1998.
- [23] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Adv. Neural Inf. Process. Syst.*, vol. 25, pp. 1097–1105, Dec. 2012.
- [24] B. Graham, "Fractional Max-pooling," *arXiv preprint arXiv:1412.6071*, pp. 1–10, Dec. 2014.
- [25] W. S. Hua, V. Govindaraj, S. L. Fernandes, Z. Zhu, and Z. Y. Dong, "Deep Rank-Based Average Pooling Network for COVID-19 Recognition," *Computers, Materials & Continua*, vol. 70, no. 2, pp. 2797–2813, Jan. 2022.

- [26] S. Kaufman, P. Phothilimthana, Y. Zhou, C. Mendis, S. Roy, A. Sabne, and M. Burrows, “A Learned Performance Model for Tensor Processing Units,” *Proceedings of Machine Learning and Systems* 3, pp.1–14, Mar. 2021.
- [27] S. Venkataramani et al., “RaPiD: AI Accelerator for Ultra-low Precision Training and Inference,” in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit.*, pp. 153–166, Jun. 2021.
- [28] S. Ryu, et al., “BitBlade: Energy-efficient Variable Bit-precision Hardware Accelerator for Quantized Neural Networks,” *IEEE J. Solid-State Circuits*, vol. 57, no. 6, pp. 1924–1935, June 2022.
- [29] B. Asgari, R. Hadidi, H. Kim, and S. Yalamanchili, “ERIDANUS: Efficiently Running Inference of DNNs Using Systolic Arrays,” *IEEE Micro*, vol. 39, no. 5, pp. 46–54, Sep./Oct. 2019.
- [30] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, “SCALE-Sim: Systolic CNN Accelerator Simulator,” *arXiv preprint arXiv:1811.02883*, pp. 1–11, Oct. 2018.
- [31] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient Processing of Deep Neural Networks: A Tutorial and Survey,” in *Proc. of the IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.
- [32] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, “Automatic Differentiation in PyTorch,” in *NIPS Workshop*, pp. 1–4, Oct. 2017.
- [33] L. Deng, “The MNIST Database of Handwritten Digit Images for Machine Learning Research,” *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 141–142, Nov. 2012.